



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

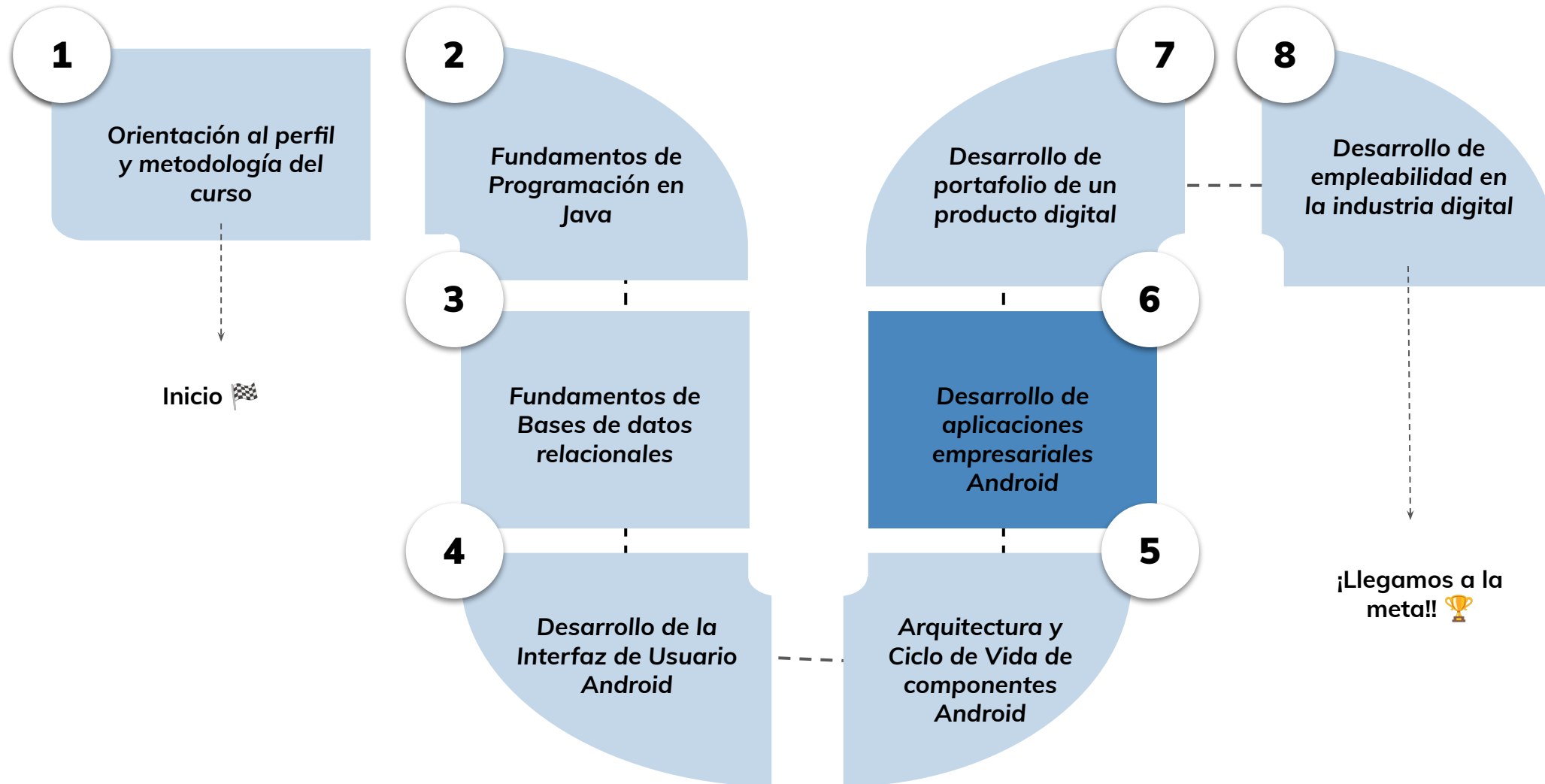


› Consumo de apis rest en android-Parte I

AE: Construir una aplicación android que consume un servicio rest actualizando la interfaz de usuario, acorde al lenguaje kotlin y a la librería retrofit.

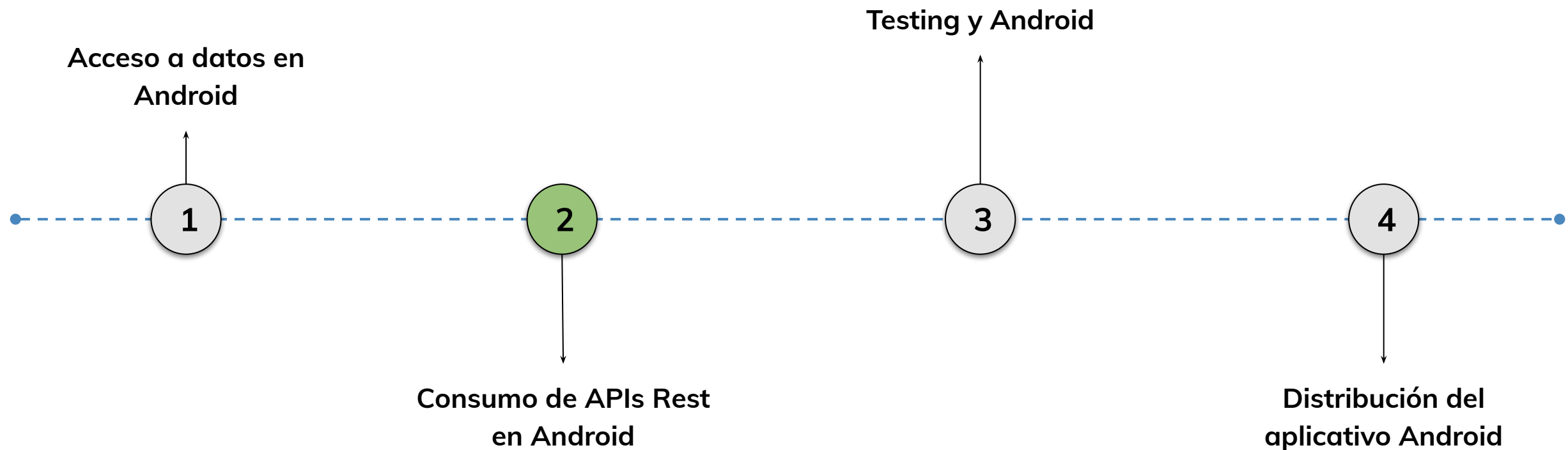
Hoja de ruta

¿Cuáles **módulos** conforman el programa?



Roadmap de la lección

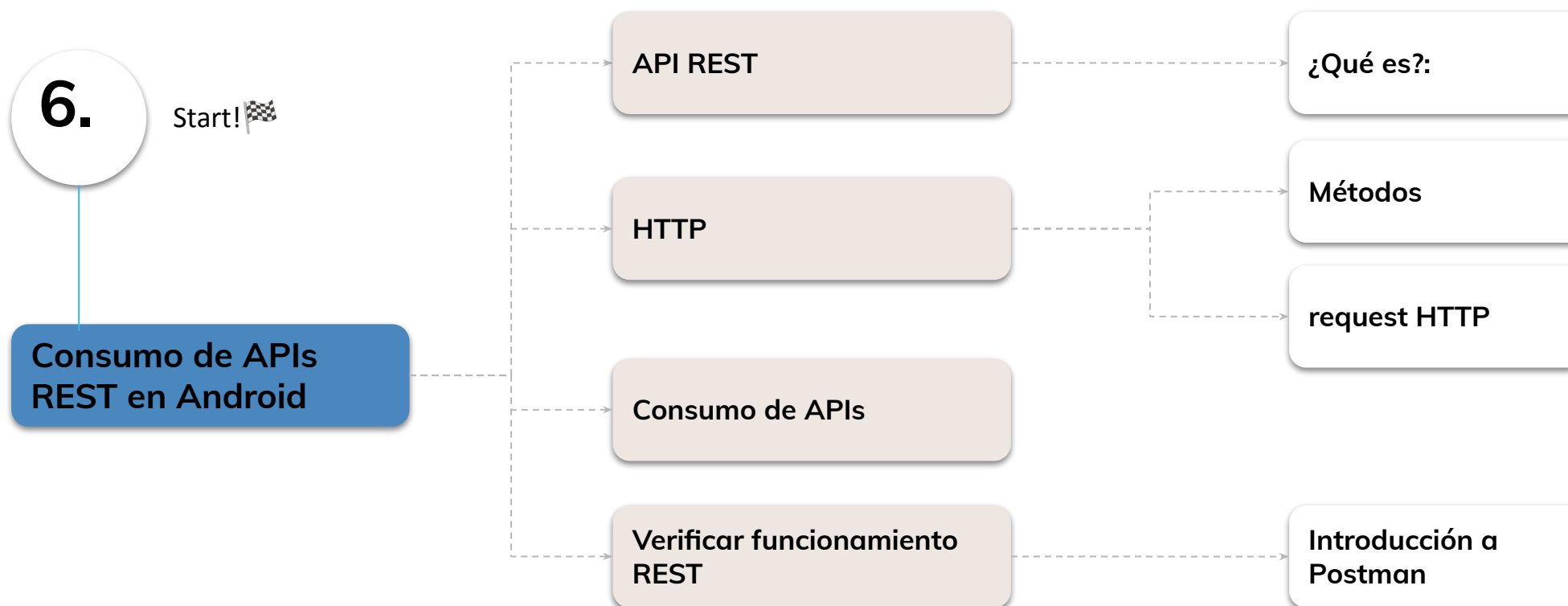
¿Cuáles *lecciones* estaremos estudiando en este módulo?





Learning Path

¿Cuáles temas trabajaremos hoy?





Objetivos de aprendizaje

¿Qué aprenderás?

- Tener una introducción a APIs REST
- Conocer mas de HTTP
- Aprender sobre consumo de APIs
- Introducción a Postman
- Hacer requests HTTP usando Postman
- Guardar, Exportar e importar en Postman
- Entender los requerimientos de la evaluación del módulo
- Comprender los pasos que vamos a tomar para implementar los cambios
- Saber más sobre los próximos pasos a seguir



Rompehielo 🥶



Contexto: 🙌

Las comunicaciones con APIs usa protocolos muy comunes en las comunicaciones web.

Consigna: ✍️

¿Qué protocolos conocen de comunicación Web?

API REST



API REST

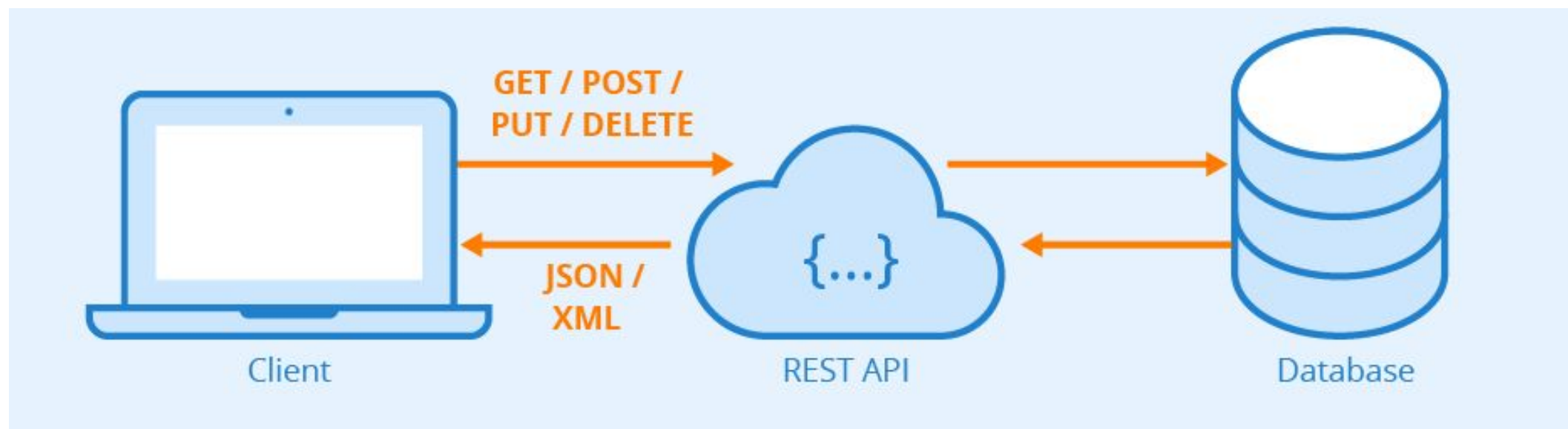


¿Qué es?:

Una API es un conjunto de reglas y protocolos que permiten a aplicaciones y dispositivos conectarse y comunicarse entre sí.

Una API REST sigue los principios de diseño de REST (Representational State Transfer), utiliza recursos con URI únicas para acceder a datos o servicios, y emplea métodos HTTP para operar sobre estos recursos. Además, enfatiza la transferencia de estado representacional, lo que significa que responde a las solicitudes proporcionando la representación actual del estado de un recurso, como datos de usuario en formato JSON o XML.

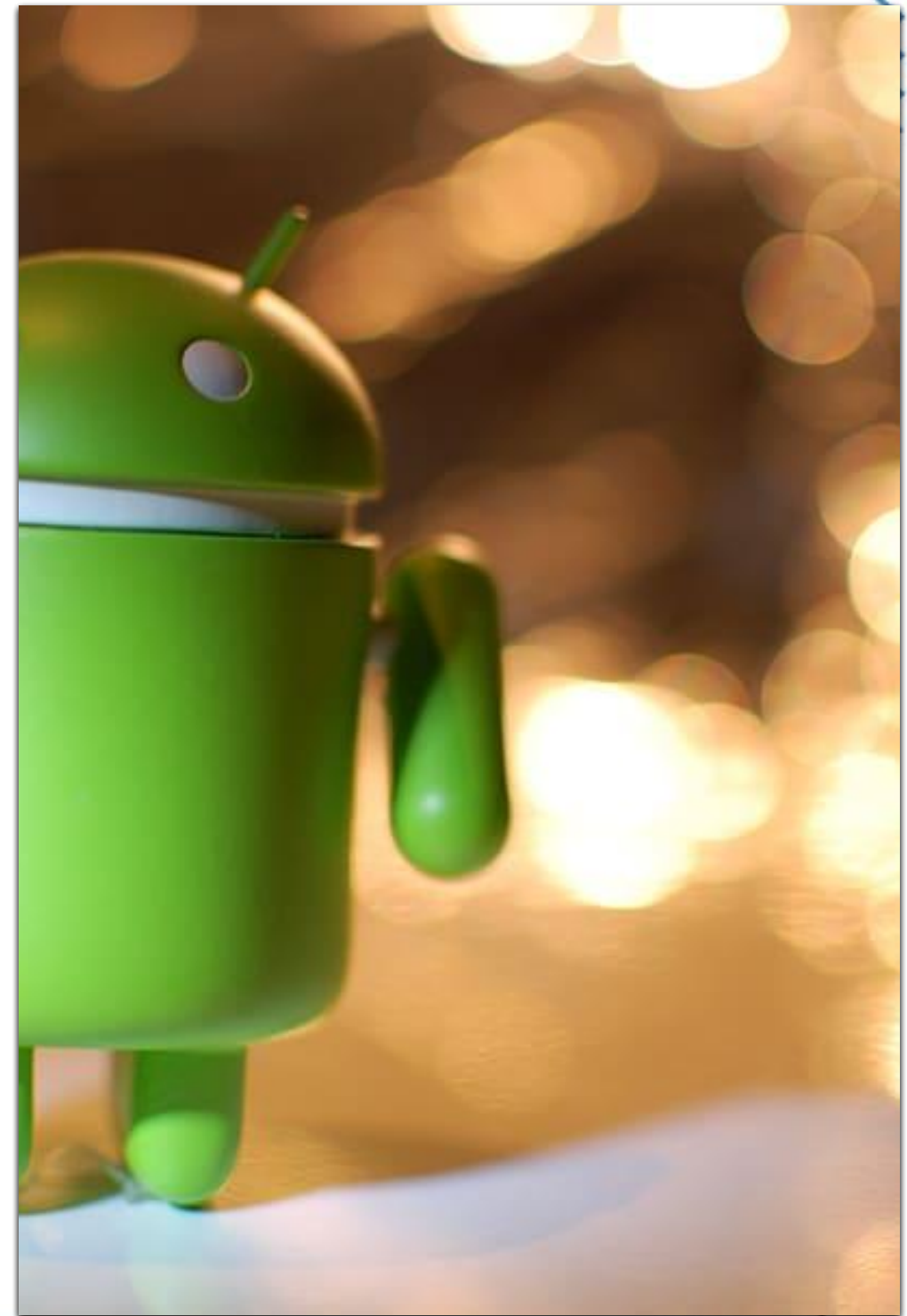
API REST



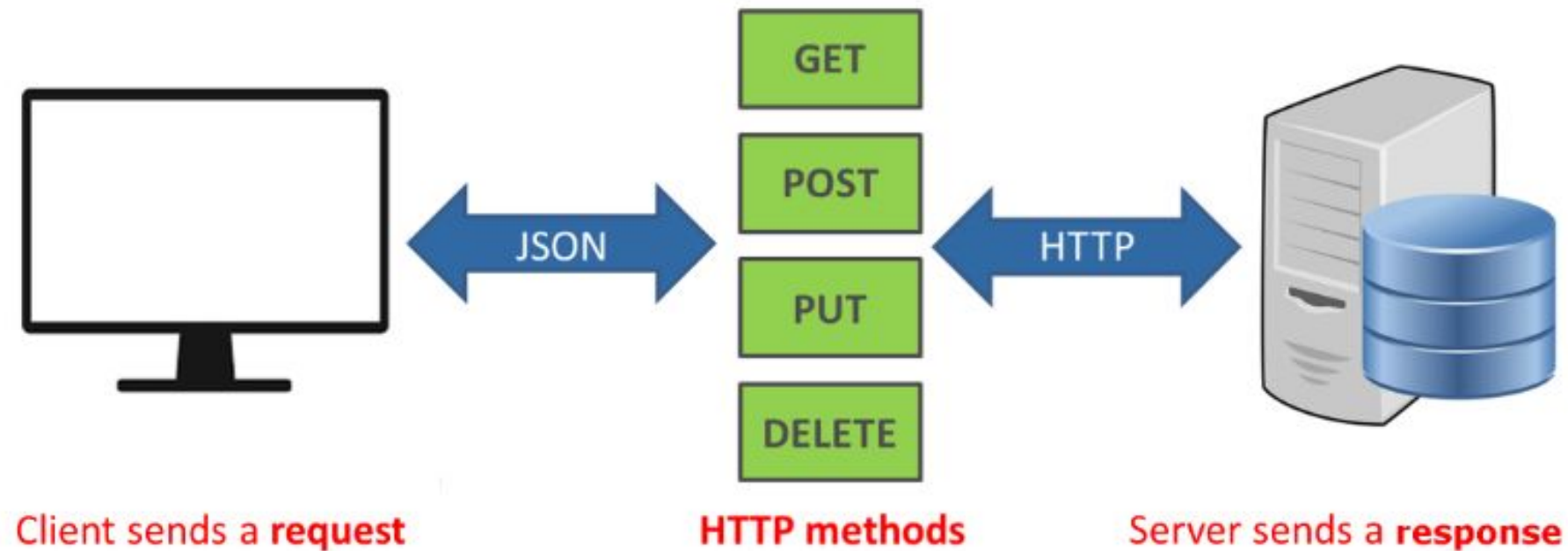
API REST

Distinción cliente/servidor:

Una API es un conjunto de reglas que permiten que una aplicación cliente se comuniquen con un servidor. El cliente utiliza la API para enviar solicitudes al servidor y acceder a los recursos, mientras que el servidor contiene los recursos y responde a las solicitudes del cliente. La API define los métodos de solicitud, los datos necesarios y el formato de las respuestas. Esto permite una clara separación entre el cliente y el servidor, permitiendo que cada uno se enfoque en sus tareas específicas sin conocer los detalles internos del otro.



Consumo de servicio API REST



ANDROID

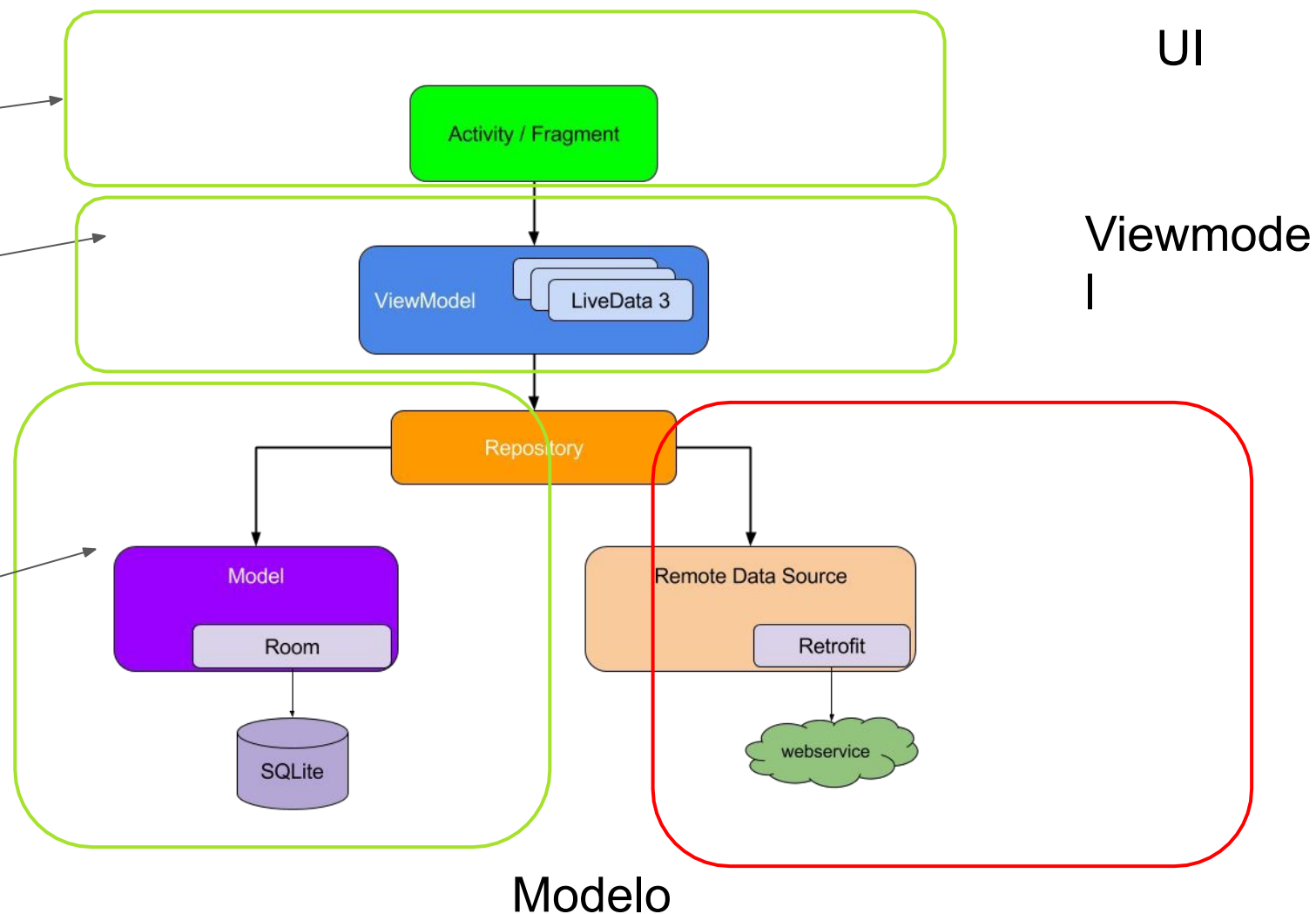


¿Que hemos hecho hasta el momento?

Esta es la capa de las vistas.

ViewModel, clase que se encargará de proporcionar la información a las vistas.

Modelo, constituido por un repositorio quien se encarga de entregar datos al viewModel. Estos datos provienen de la base de datos administrada por Room o desde una fuente de datos externa.



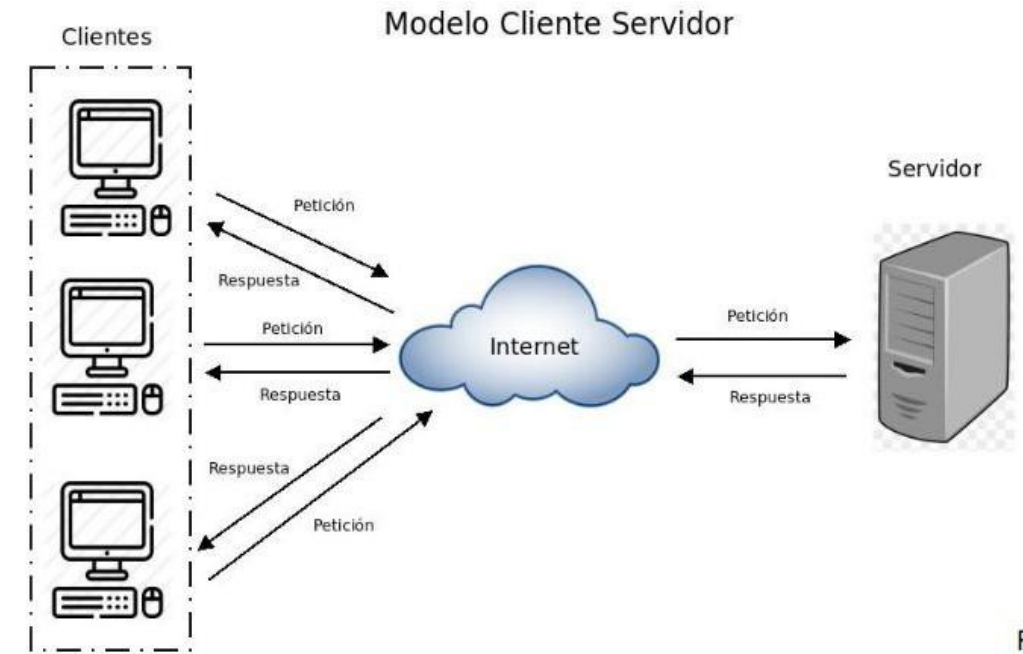
Cliente Servidor



Cliente - Servidor

En las arquitecturas cliente-servidor web, se consideran los siguientes elementos:

- Un conjunto específico de estándares de comunicación, también llamados protocolos para la interacción entre sistemas. Los más populares HTTP y HTTPS (Protocolo de transferencia de hipertexto).
- Estructura de datos de intercambio, nos referimos a la estructura de los datos formateados, por ejemplo JSON o XML.

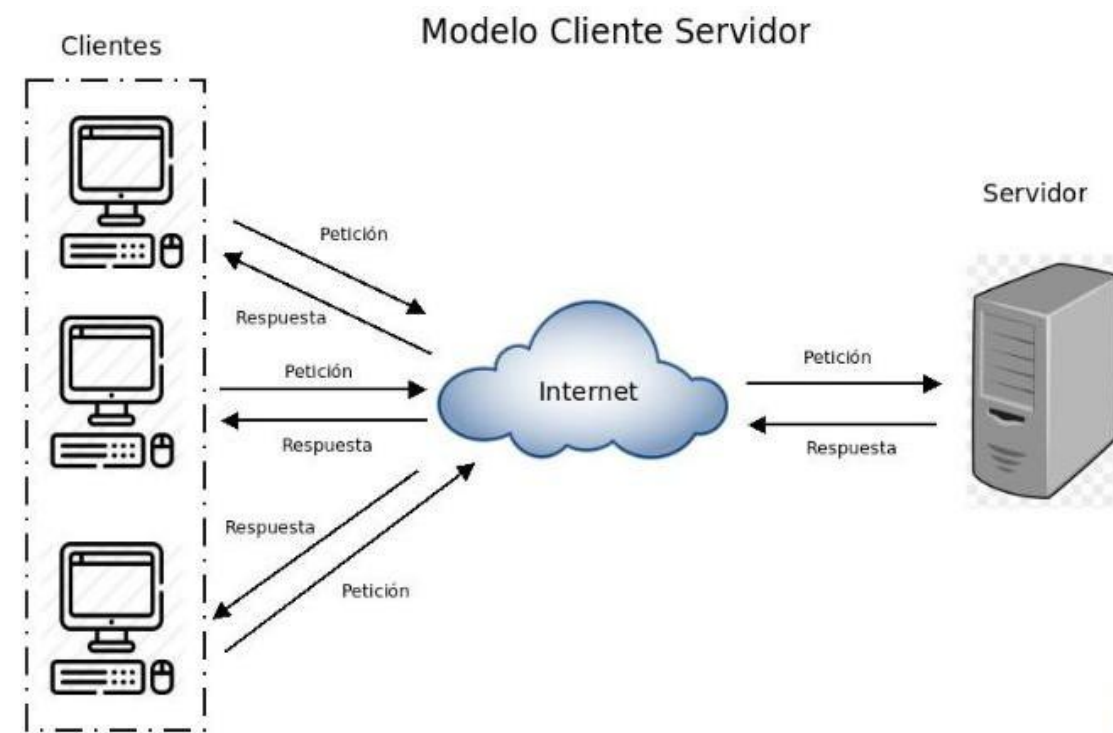


01. Esquema del Modelo Cliente Servidor

Figura

CLIENTE

- Iniciar Request o solicitud de datos
- Esperar y recibir respuestas
- Conectarse a uno o varios servidores
- Exponer datos a los usuarios en alguna UI.



01. Esquema del Modelo Cliente Servidor

Figura



SERVIDOR

- Esperar los requerimientos de los clientes.
- Recibir, procesar, preparar y enviar respuestas.
- Aceptar múltiples conexiones.
- No interactúa con usuarios finales.

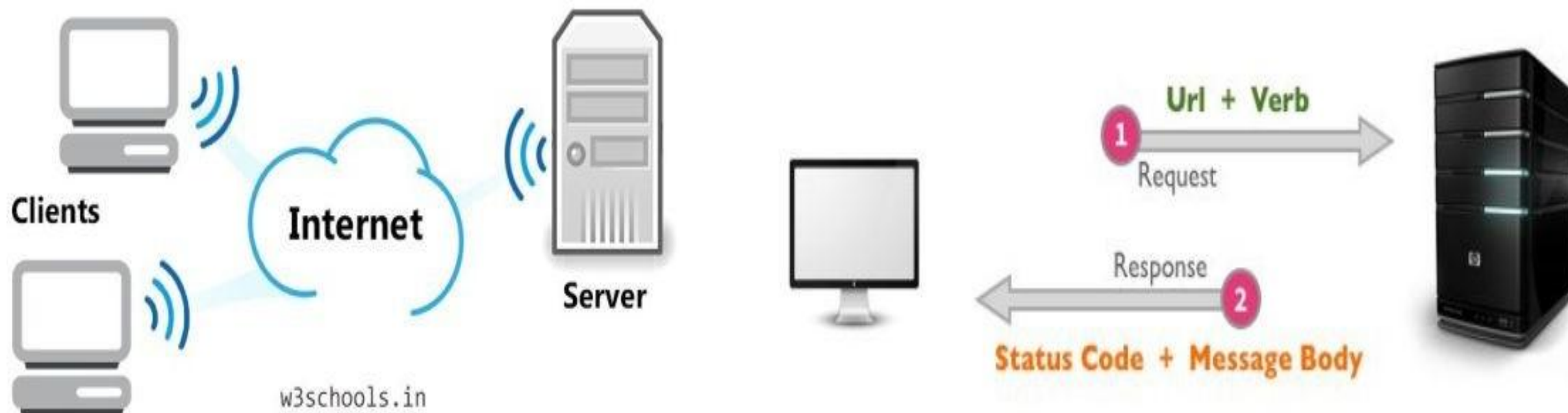


CLIENTE SERVIDOR

- Conjunto específico de idiomas o normas para comunicarse.
- Mecanismo o protocolo para solicitar aspectos requeridos a un servidor.
- Respuestas del servidor enviado en formato estándar de comunicación (Json , Text, XML)



EJEMPLOS DE ARQUITECTURA CLIENTE



Servicios REST



API

API proviene del inglés (**A**pplication **P**rogramming **I**nterface), o interfaz de programación de aplicaciones. Es el conjunto de rutinas, funciones y procedimientos o métodos que nos permiten utilizar recursos de un software, desde otro, sirviendo como una capa de abstracción o intermediario.

En otras palabras podemos decir que es un método generalizado para consumir un servicio.



API- API REST

Expone recursos a través de una o distintas direcciones web utilizando arquitectura basada en HTTP.

{ REST:API }



PRINCIPIOS DE REST



Cliente-Servidor

Sin estado
(Stateless)

Caché

Interfaz uniforme

Sistema de capas

Código bajo la
demanda



REST-CRUD

C

Create - crear

R

Read - leer

U

Update - actualizar

D

Delete - borrar

➤ HTTP: *Introducción*



/* HTTP Requests */

HTTP Requests HTTP (Protocolo de transferencia de hipertexto) es un protocolo utilizado para transmitir datos a través de Internet. Es la base de la World Wide Web y se utiliza para la comunicación entre navegadores web y servidores web. Una solicitud HTTP es un mensaje enviado por un cliente (como un navegador web) a un servidor (como un servidor web) para solicitar un recurso o servicio específico. La solicitud contiene varios componentes, que incluyen:

A. Un método: la acción que el cliente quiere realizar. Los métodos más utilizados son GET, POST, PUT y DELETE.

B. Un URI: la ubicación del recurso o servicio al que el cliente quiere acceder.

C. Encabezados: información adicional sobre la solicitud, como el tipo de contenido que se envía, el idioma preferido y las credenciales de autenticación.

D. El cuerpo de un mensaje: datos que el cliente envía al servidor, como el envío de un formulario o una carga JSON



/* HTTP Requests */



Este es un ejemplo de una solicitud HTTP GET:

```
GET /index.html HTTP/1.1 Host: www.example.com  
Accept-Language: en-US User-Agent: Mozilla/5.0
```

La primera línea de la solicitud se denomina línea de solicitud y contiene el método (GET), el URI (index.html) y la versión de HTTP que se utiliza (HTTP/1.1). Las líneas restantes son encabezados que brindan información adicional sobre la solicitud.



/* HTTP Requests */

Cuando el servidor recibe una solicitud HTTP, la procesa y envía una respuesta HTTP al cliente, la respuesta contiene un **código de estado**, encabezados y un cuerpo de mensaje. El código de estado indica el resultado de la solicitud, como éxito (200 OK) o falla (404 No encontrado).





/* HTTP Requests */

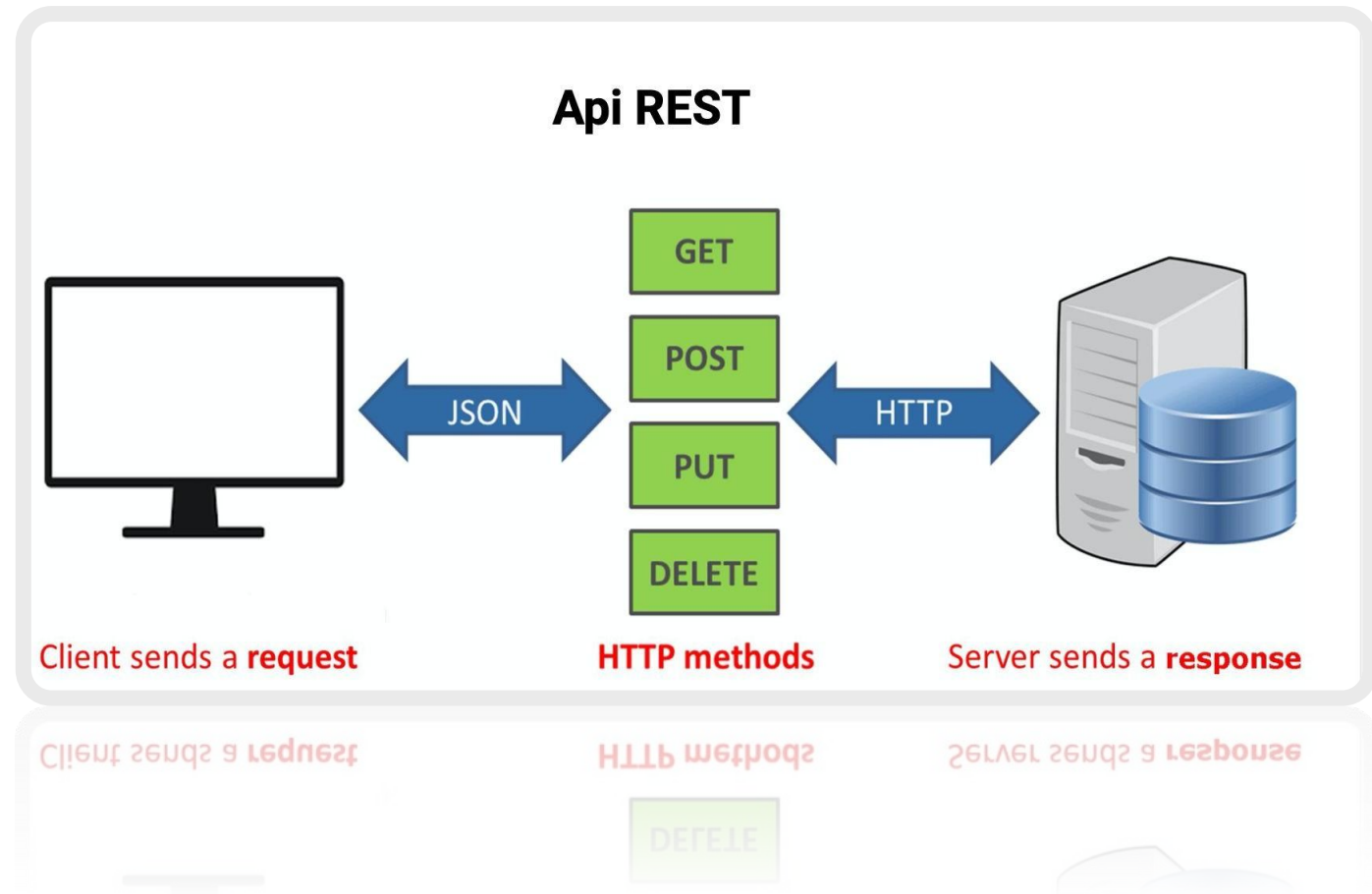


Las solicitudes y respuestas HTTP generalmente se envían a través de **conexiones TCP/IP** y no tienen estado, lo que significa que el servidor no retiene ninguna información sobre el cliente entre solicitudes. Esto permite **escalabilidad y flexibilidad en el sistema**, pero también significa que se pueden necesitar mecanismos adicionales, como **cookies y sesiones**, para mantener interacciones con el estado de la conexión.

VERBOS HTTP

VERBOS REST

- GET
- POST
- PUT
- PATCH
- DELETE
- HEAD





VERBOS REST



Verbos HTTP - (HTTP Methods) Los verbos HTTP, también conocidos como métodos HTTP, son los comandos que un cliente envía a un servidor para solicitar una acción específica. Los verbos HTTP más utilizados son:

- **GET:** solicita que el servidor devuelva la representación actual de un recurso específico. Este es el verbo más común utilizado para recuperar datos de un servidor.
- **POST:** solicita que el servidor acepte la representación adjunta y la almacene como un nuevo recurso. Este verbo se usa para enviar datos al servidor, como cuando se completa un formulario o se carga un archivo.



VERBOS REST

- **PUT:** solicita que el servidor acepte la representación adjunta y la almacene como el recurso especificado. Este verbo es similar a POST, pero normalmente se usa para actualizar un recurso existente.
- **DELETE:** solicita que el servidor elimine el recurso especificado.
- **HEAD:** igual que GET pero devuelve solo los encabezados y no el cuerpo de la respuesta.
- **OPTIONS:** solicita que el servidor devuelva los métodos HTTP que admite.
- **PATCH:** solicita que el servidor aplique un conjunto de modificaciones al recurso especificado.



VERBOS REST

Todos estos verbos son parte de la especificación HTTP/1.1, y algunos otros verbos como CONNECT, TRACE y otros que existen, pero no se usan mucho. Estos verbos indican la acción que se desea realizar en el recurso identificado. Mediante el uso del verbo apropiado, los clientes pueden crear, leer, actualizar y eliminar recursos en el servidor.



Es importante tener en cuenta que no todos los servidores admiten todos los verbos y que diferentes verbos pueden tener diferentes comportamientos o restricciones según el servidor.

PARTES DE UN REQUEST HTTP



PARTES DE UN REQUEST

Encabezados

Información adicional sobre la solicitud, como el tipo de contenido que se envía, el idioma preferido y las credenciales de autenticación.

1

2

3

Línea de solicitud

Primera línea de la solicitud que contiene el método, el URI y la versión de HTTP que se está utilizando.

Cuerpo del mensaje

Datos que el cliente envía al servidor, como el envío de un formulario o una carga JSON. No todas las solicitudes tienen un cuerpo de mensaje



PARTES DE UN REQUEST



Utilicemos el ejemplo anteriormente mostrado:

GET /index.html HTTP/1.1 Host: www.example.com
Accept-Language: en-US User-Agent: Mozilla/5.0 En este
ejemplo, la línea de solicitud es "GET /index.html
HTTP/1.1",

los encabezados son "Host: www.example.com",
"Accept-Language: en-US" y "User-Agent: Mozilla/5.0" y
no hay cuerpo de mensaje en esta solicitud



PARTES DE UN REQUEST

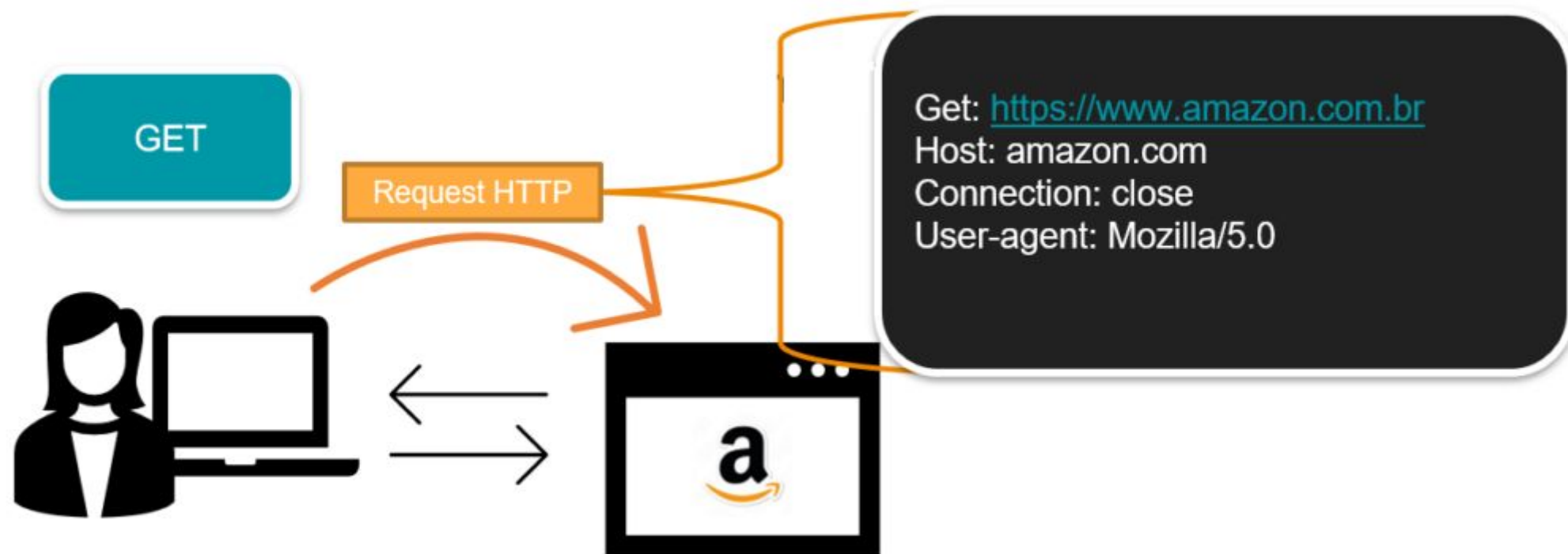
La línea de Solicitud se compone de:

- **Método:** La acción que el cliente quiere realizar. Los métodos más utilizados son GET, POST, PUT y DELETE.
- **URI:** La ubicación del recurso o servicio al que el cliente quiere acceder.
- **Versión HTTP:** la versión del protocolo que se está utilizando. HTTP/1.1 es la versión más común en uso.
- **Los encabezados o Headers:** proporcionan información adicional sobre la solicitud. Son pares clave-valor separados por dos puntos. Los encabezados pueden incluir información como el tipo de contenido, el agente de usuario, el host, la codificación y más.

El cuerpo del mensaje se usa para enviar datos al servidor, puede contener datos como parámetros de formulario, carga útil JSON y más. El cuerpo del mensaje no está presente en todas las solicitudes, por ejemplo, la solicitud GET no tiene un cuerpo de mensaje.



HTTP



HTTP



CÓDIGOS DE RESPUESTA





CÓDIGOS DE RESPUESTA



Los códigos de respuesta HTTP, también conocidos como códigos de estado HTTP, son números de tres dígitos que devuelve un servidor en respuesta a una solicitud HTTP. Estos códigos indican el resultado de la solicitud y brindan información adicional sobre el estado del recurso solicitado.

El primer dígito del código de estado define la clase de respuesta. Las clases más comunes son:

1xx (Informativo): Se recibió la solicitud, proceso continuo.

- **2xx (Successful):** la solicitud se recibió, comprendió y aceptó correctamente.

- **3xx (Redireccionamiento):** la solicitud necesita más acciones para completarse, como redirigir al cliente a una ubicación diferente.

- **4xx (Error del cliente):** la solicitud contiene una sintaxis incorrecta o el servidor no puede cumplirla.

- **5xx (Error del servidor):** el servidor no pudo cumplir con una solicitud válida



CÓDIGOS DE RESPUESTA



Códigos de respuesta Algunos de los códigos de respuesta HTTP más comunes son:

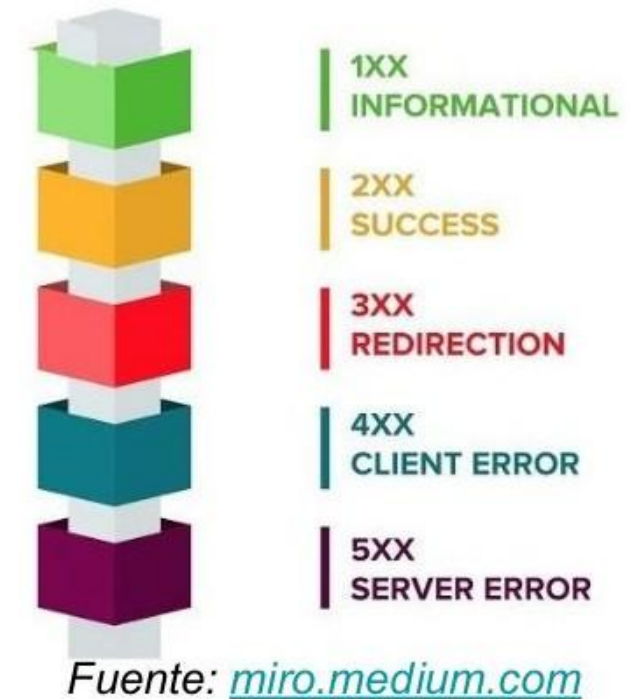
- 200 OK: La solicitud fue exitosa y el servidor ha devuelto la información solicitada.
- 201 Creado: La solicitud fue exitosa y el servidor ha creado un nuevo recurso.
- 204 Sin contenido: la solicitud fue exitosa pero no hay información para devolver.
- 400 Solicitud incorrecta: la solicitud contiene una sintaxis incorrecta o el servidor no puede cumplirla.
- 401 No autorizado: la solicitud requiere autenticación.
- 403 Prohibido: El servidor entiende la solicitud pero se niega a autorizarla.
- 404 No encontrado: No se pudo encontrar el recurso solicitado.
- 500 Error interno del servidor: se produjo un error en el servidor y no se pudo completar la solicitud. Estos códigos son solo algunos ejemplos de los muchos códigos de respuesta HTTP que existen. Cada código representa un significado específico y es importante manejarlos de la manera adecuada.



Es importante tener en cuenta que estos códigos son un estándar y no todos los servidores devolverán los mismos códigos para los mismos errores, además, se pueden agregar nuevos códigos a medida que evoluciona el estándar.

HTTP response Codes

- 200 - 299 Indica que la solicitud fue exitosa
- 300 - 399 Indica que la solicitud se realizó, Pero debe realizar otra acción todavía.
- 400 - 499 Error donde regularmente no se encuentran los recursos
- 500- 599 Regularmente error en el servidor



JSON

JSON

JSON es el acrónimo para
JavaScript Object
Notation



■ Estructura de un archivo JSON

La estructura de un archivo JSON se puede agrupar en dos tipos:

- Colección de Clave-valor (key-value) lo cual puede ser interpretado como un objeto en los distintos tipos de lenguajes de programación. También son conocidos como diccionario o tabla Hash, es una lista con clave valor.
- Una lista ordenada de valores, esto en la mayoría de los lenguajes de programación podría ser como un arreglo, vector, listado o secuencia.

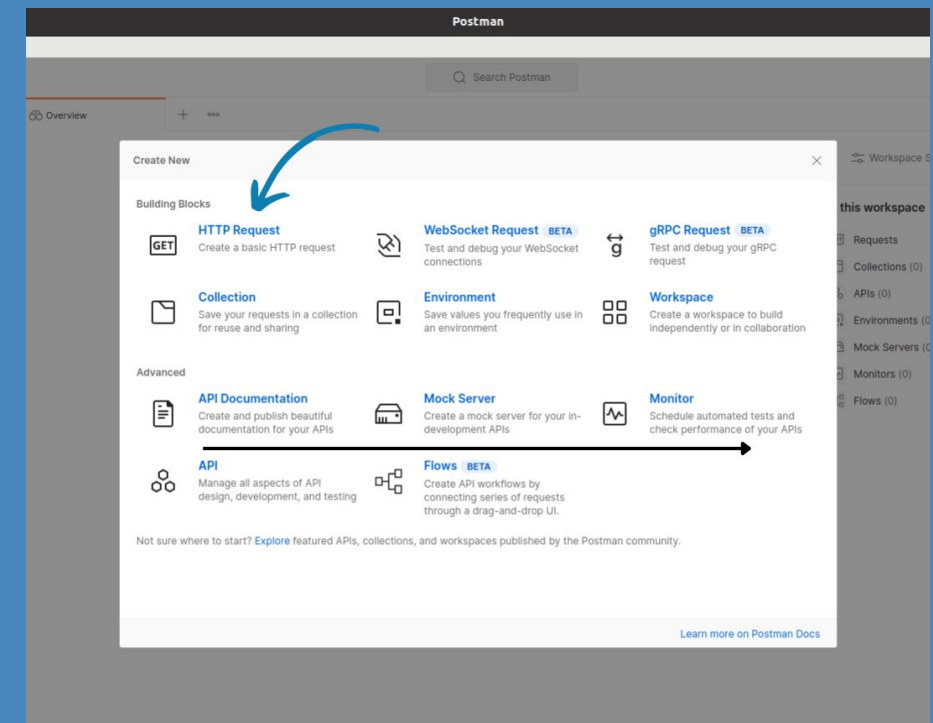
```
[  
  {  
    "price": 450000,  
    "id": "424905",  
    "type": "buy",  
    "img_src":  
      "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0044631300503690E01\_DXXX.jpg"  
  },  
  {  
    "price": 8000000,  
    "id": "424906",  
    "type": "rent",  
    "img_src":  
      "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631300305227E03\_DXXX.jpg"  
  }  
]
```

› Hacer requests HTTP usando Postman

Hacer requests HTTP usando Postman

Crea una nueva solicitud:

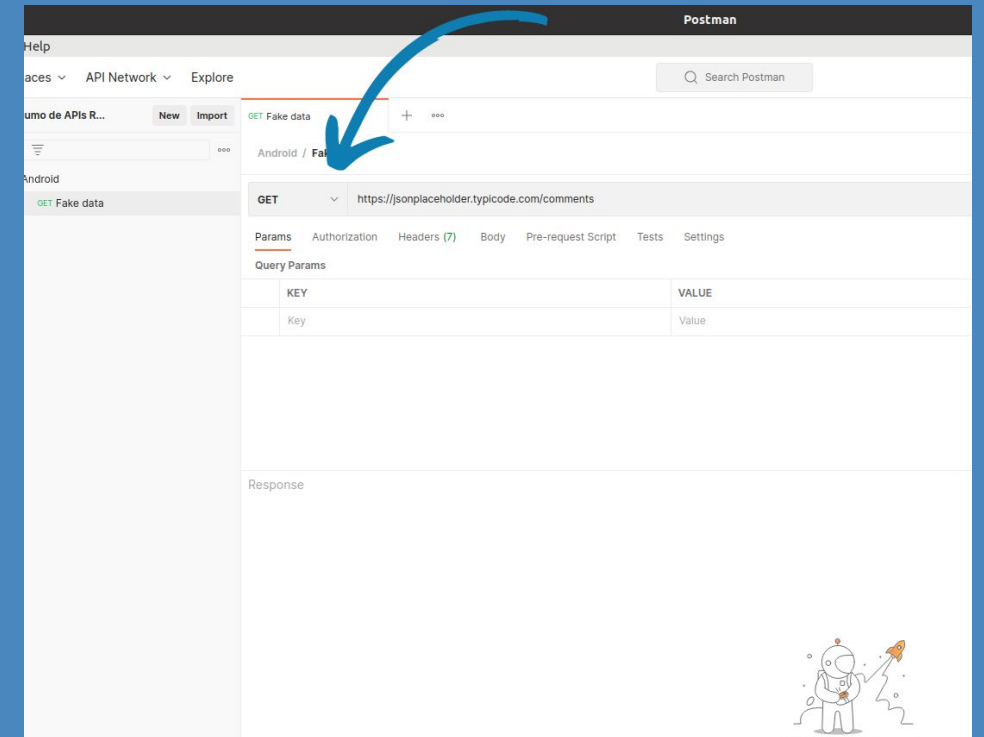
Clickeá en el botón "Nuevo" para crear una nueva solicitud.



Hacer requests HTTP usando Postman

Configura la solicitud:

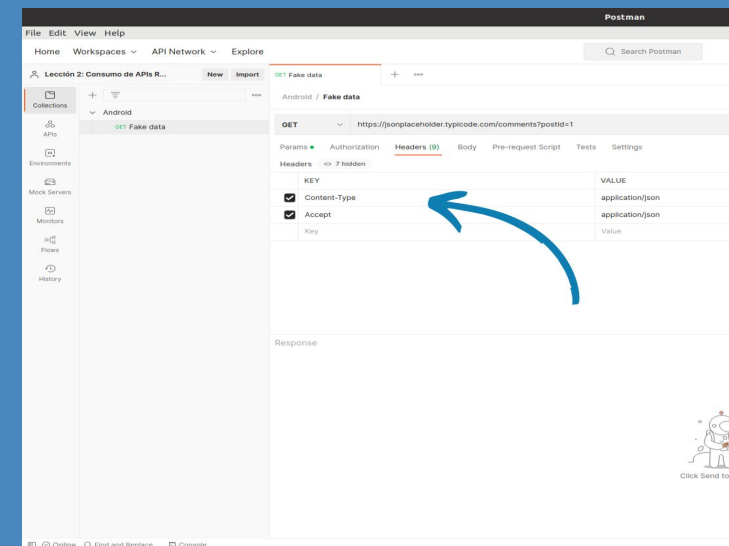
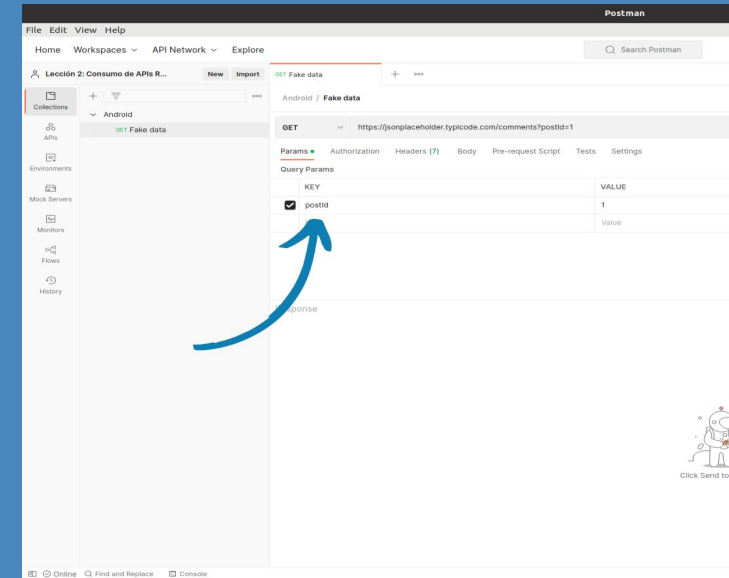
Al crear una nueva solicitud en Postman, proporciona la URL del endpoint y selecciona el método HTTP apropiado en el menú desplegable. Asegúrate de incluir cualquier ruta, parámetros o encabezados necesarios en la URL.



Hacer requests HTTP usando Postman

Agrega parámetros y encabezados:

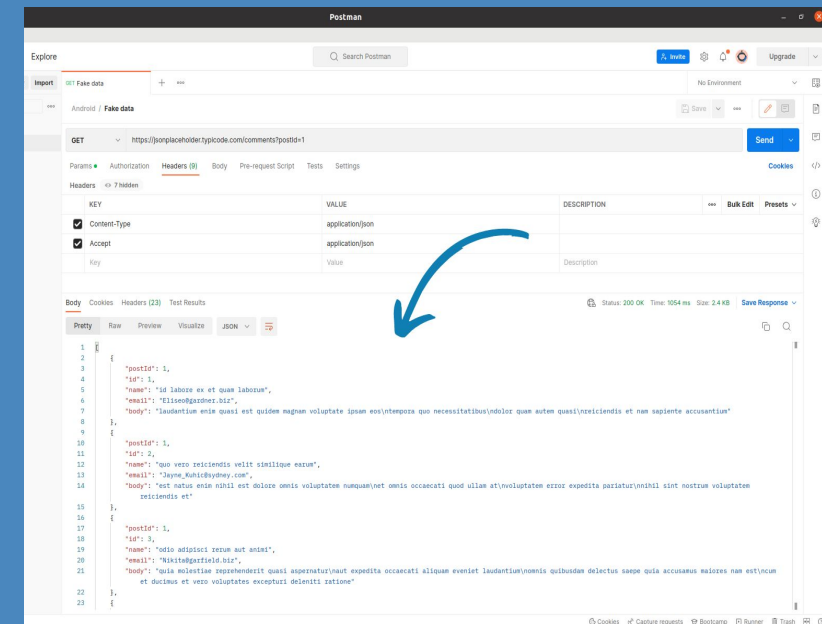
Para añadir parámetros o encabezados adicionales a tu solicitud en Postman, utiliza las pestañas "Parámetros" y "Encabezados" debajo de la URL. En la pestaña de Parámetros, puedes incluir parámetros clave-valor para enviar en la URL o en el cuerpo de la solicitud, según el método HTTP. La pestaña de Encabezados te permite agregar encabezados personalizados, útiles para la autenticación o el envío de tokens y otros requisitos específicos de la API.



Hacer requests HTTP usando Postman

Envía la solicitud:

Después de configurar todos los detalles de tu solicitud en Postman, basta con hacer clic en "Enviar". Postman enviará la solicitud al servidor y mostrará la respuesta en la sección de Respuesta, incluyendo el código de estado HTTP, los encabezados y el cuerpo de la respuesta.



› Guardar, Exportar e importar en Postman

Postman

Postman en sus inicios nace como una extensión que podía ser utilizada en el navegador Chrome de Google y básicamente nos permite realizar peticiones de una manera simple para testear **APIs** de tipo **REST** propias o de terceros. Gracias a los avances tecnológicos, Postman ha evolucionado y ha pasado de ser de una extensión a una aplicación que dispone de herramientas nativas para diversos sistemas operativos como lo son Windows, Mac y Linux. Cuenta con una versión libre de pago y con tres planes (básico, profesional y empresarial), si deseas consultar el detalle entre cada plan y sus precios puedes verlo en su [web oficial](#).



Para qué sirve Postman?

Postman sirve para múltiples tareas dentro de las cuales destacaremos en esta oportunidad las siguientes:

- **Testear** colecciones o catálogos de APIs tanto para Frontend como para Backend.
- **Organizar en carpetas**, funcionalidades y módulos los servicios web.
- Permite gestionar el ciclo de vida (conceptualización y definición, desarrollo, monitoreo y mantenimiento) de nuestra **API**.
- **Generar documentación** de nuestras APIs.
- **Trabajar con entornos** (calidad, desarrollo, producción) y de este modo es posible compartir a través de un entorno **cloud** la información con el resto del equipo involucrado en el desarrollo.



MÉTODOS MÁS UTILIZADOS Y POSIBLES ERRORES

Postman cuenta con una serie de métodos que nos permiten tomar acción ante nuestras peticiones, los más utilizados:

- **GET:** Obtener información
- **POST:** Agregar información
- **PUT:** Reemplazar la información
- **PATCH:** Actualizar alguna información
- **DELETE:** Borrar información

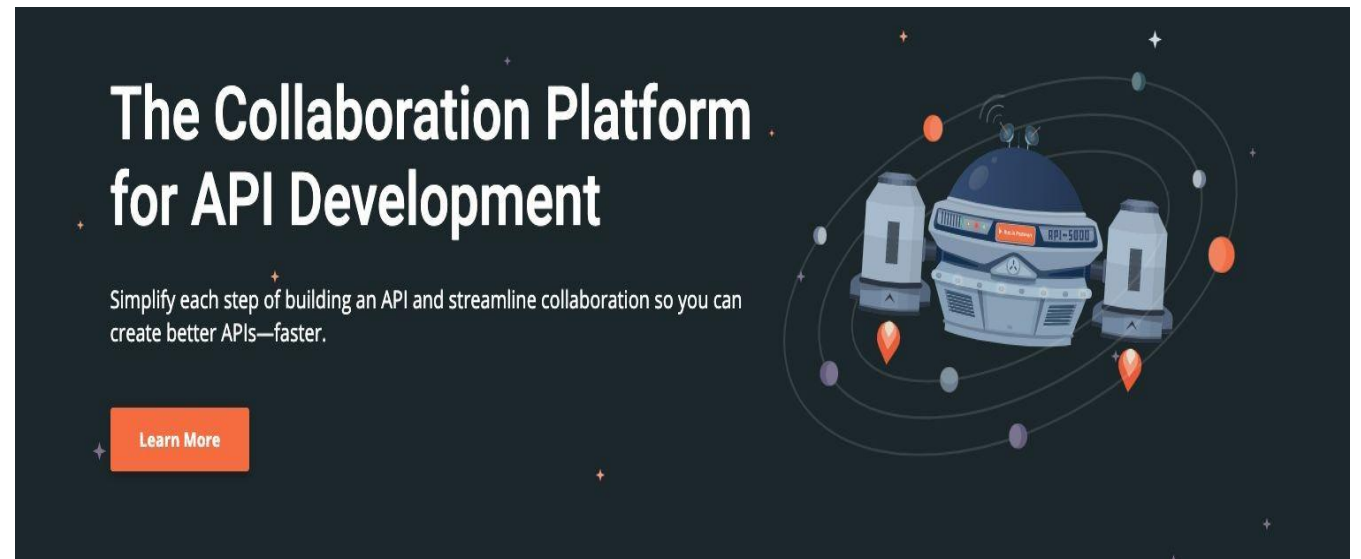


En cuanto a los posibles errores que podemos apreciar en la respuesta que nos ofrece la herramienta, lo resumiremos en que si la respuesta dada se encuentra en el rango de “200” quiere decir que toda la petición ha salido sin inconvenientes; mientras que el rango de los códigos de error “400” hacen referencia a errores con el cliente y aquellos errores en la línea de los “500” tienen que ver con fallos en el servidor.



Descargando Postman

<https://www.getpostman.com/downloads/>



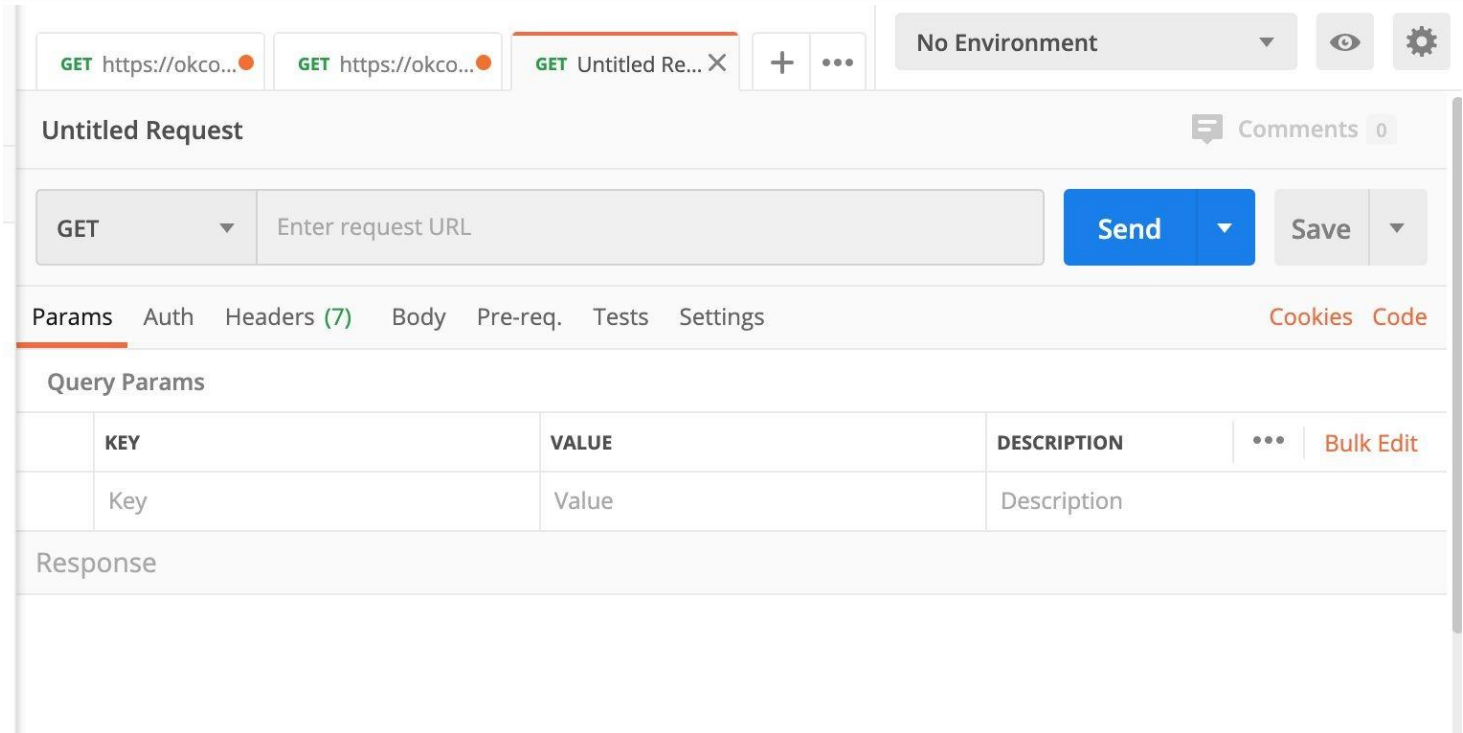
**The Collaboration Platform
for API Development**

Simplify each step of building an API and streamline collaboration so you can create better APIs—faster.

[Learn More](#)

The banner features a central illustration of a blue robot with a dome-shaped head and a screen on its chest displaying 'API-5000'. The robot is flanked by two grey vertical units. They are surrounded by several orange and blue circular markers connected by thin lines, suggesting a network or orbit. The background is dark blue with small white stars.

1. Conociendo Postman.



2. Realizando nuestro primer GET

<https://jsonplaceholder.typicode.com/>



< >
< >
< >
< >
< >
< >
< >



LIVECODING HACIENDO UN POST

3. Simular un POST

<https://jsonplaceholder.typicode.com/>



LIVECODING ANALIZANDO NUESTRA RESPUESTA

4. Analizar respuesta

- <https://jsonplaceholder.typicode.com/>
- <https://mindicador.cl/>

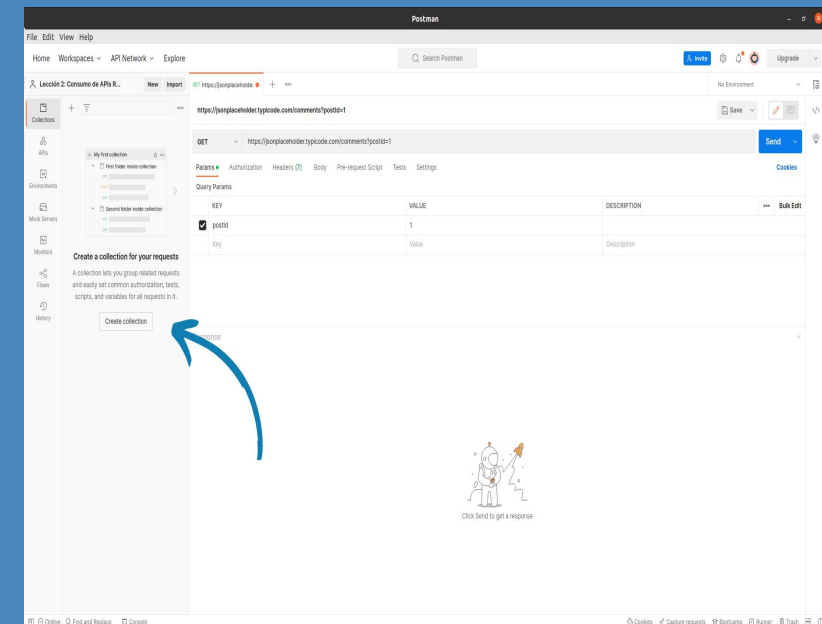


Guardar

En Postman, puedes organizar y reutilizar tus solicitudes guardándolas en colecciones.

Crear una colección:

Dirígete a la barra lateral izquierda de la interfaz, haz clic en "Crear colección", proporciona un nombre descriptivo y guarda la colección.



Guardar

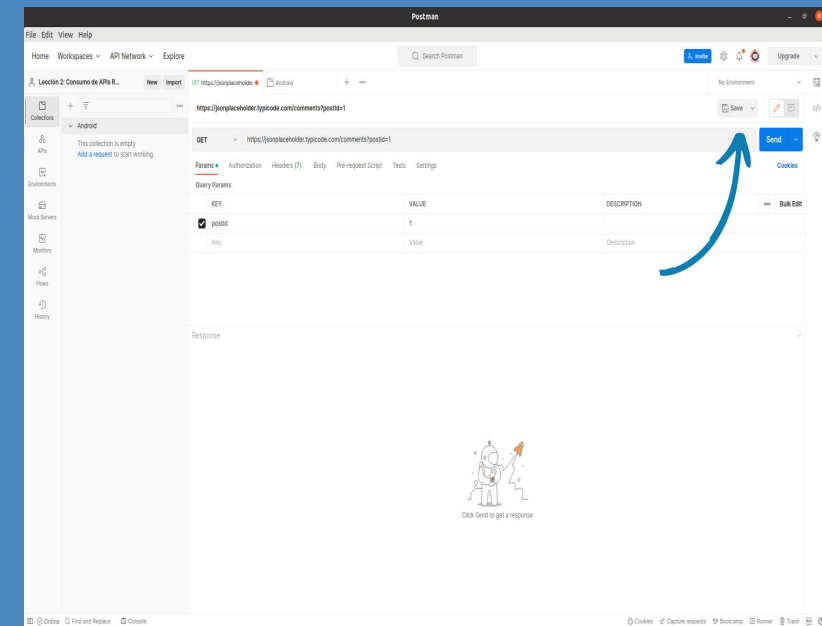
Guardar un request en la colección:

Abre la solicitud que deseas guardar en la colección.

En la esquina superior derecha del panel de solicitud, haz clic en el botón con el ícono de "Guardar en la colección".

Se abrirá una ventana emergente donde podrás configurar los detalles del request guardado en la colección.

En la sección "Guardar en", asegúrate de que la colección correcta esté seleccionada.



Guardar

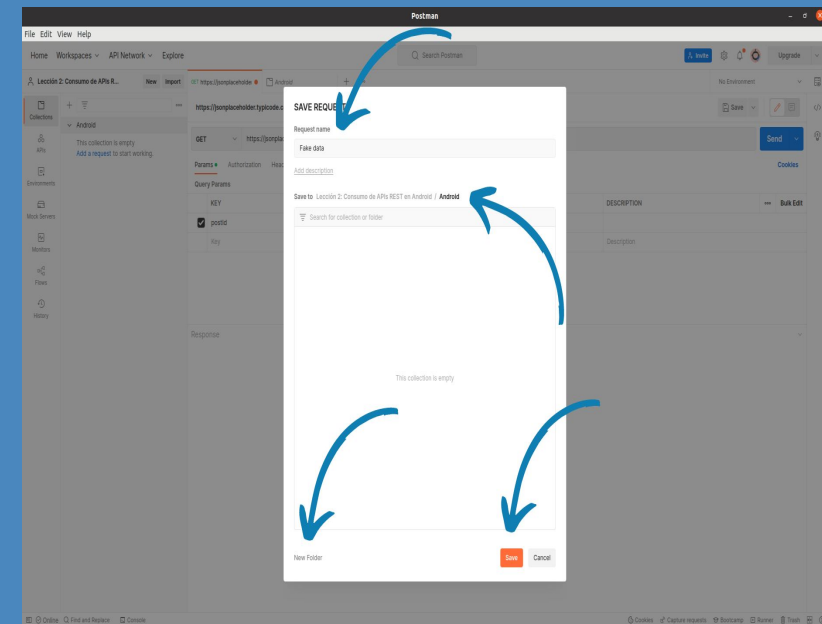
Guardar un request en la colección:

Proporciona un nombre descriptivo en el campo "Nombre".

Opcionalmente, agrega una descripción más detallada en el campo "Descripción".

Si deseas guardar el request en una carpeta específica dentro de la colección, selecciona la carpeta adecuada en el menú desplegable "Carpeta".

Una vez que hayas configurado los detalles, haz clic en el botón "Guardar" para guardar el request en la colección.





Exportar e importar

¿Cómo?:

Postman permite exportar e importar solicitudes, colecciones y entornos, facilitando la colaboración y el intercambio de configuraciones entre usuarios y equipos.

Exportar e importar

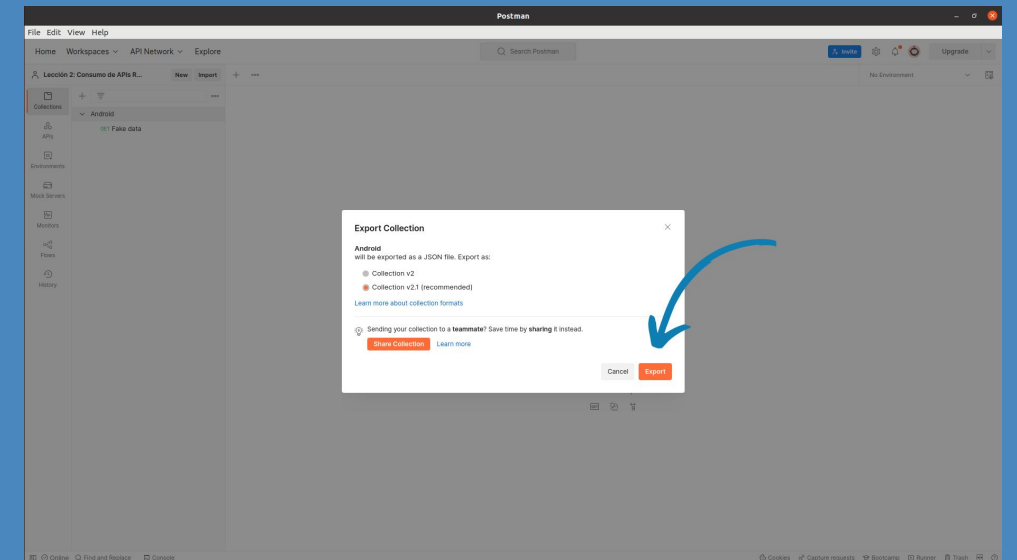
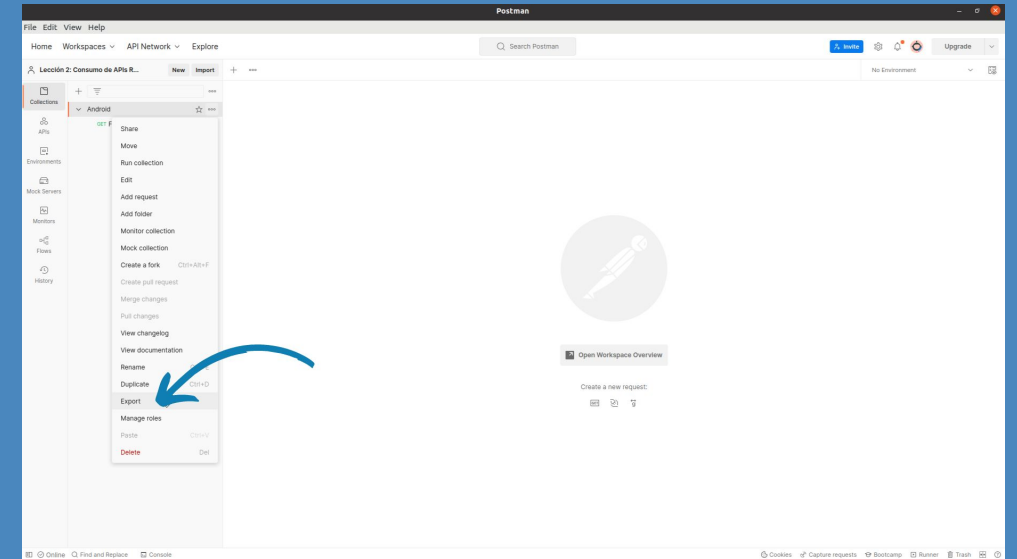
Exportar:

En la interfaz principal, selecciona el elemento que deseas exportar en la barra lateral izquierda.

Haz clic con el botón derecho del ratón y elige "Exportar".

Selecciona el formato de exportación deseado.

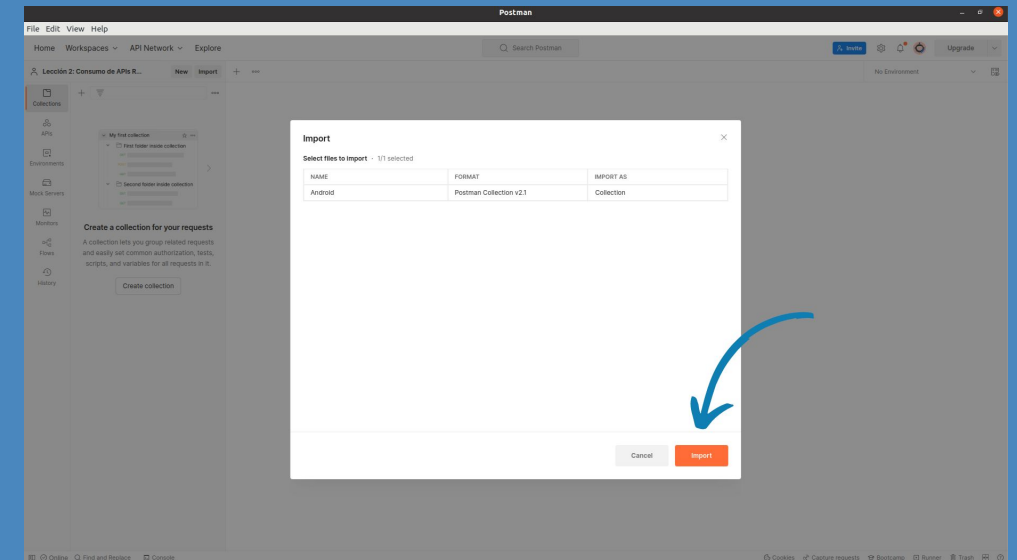
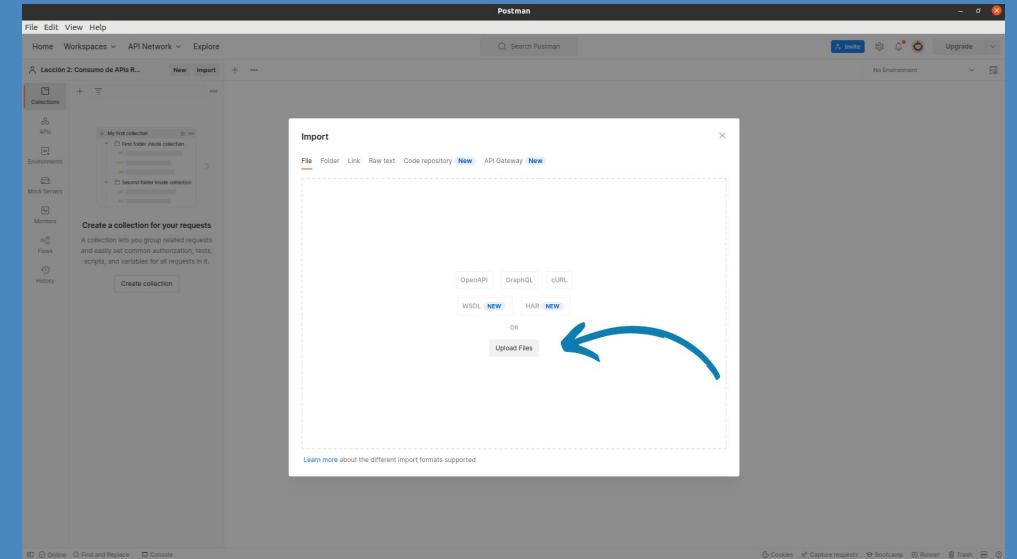
Especifica la ubicación en tu dispositivo donde deseas guardar el archivo de exportación y haz clic en "Guardar".



Exportar e importar

Importar:

Abre Postman y selecciona el botón "Importar" en la barra lateral izquierda. Busca en tu dispositivo el archivo que deseas importar y selecciónalo. Haz clic en "Importar" para que Postman cargue los datos del archivo seleccionado en tu entorno de trabajo.



<>
<>
<>
<>
<>
<>
<>
<>
<>
<>



LIVECODING GUARDANDO UNA COLECCIÓN

5. Guardar una colección en Postman

- <https://jsonplaceholder.typicode.com/>
- <https://mindicador.cl/>





Momento:

Time-out!



5 -10 min.



LIVE CODING

Ejemplo en vivo

Practica con Postman:

Mostrar cómo hacer una llamada con GET a la API de ejemplo “<https://jsonplaceholder.typicode.com/users>” y ver la respuesta dentro de POSTMAN.

Tiempo: 20 min

MÓDELAR JSON ORIENTADO A OBJETOS



GSON o MOSHI

En algún momento necesitaremos transformar de JSON a Objeto, Esto lo realizaremos con la ayuda de alguna librería como:

- [Gson](#)
- [Moshi](#)

Retrofit tiene una implementación para trabajar con cualquier elección escogida.

Data class (POJO)

A través de una data class podremos indicar los campos que vamos a transformar a objetos. Estos deben ser una representación de los datos a recibir en Json.

La anotación `@SerializedName` permite asociar nombres con variables.

```
data class Recipe
    (val title: String,
    val version: Double,
    val href: String,
    @SerializedName("result_list")
    val results: List<Results>
    )
```

› Retrofit

Retrofit

¿Qué es?:

Una biblioteca de Square para Android y Java que simplifica la interacción con servidores REST. Ofrece una capa de abstracción para la comunicación de red y las solicitudes HTTP, facilitando a los desarrolladores la interacción eficiente con servicios web.





Cliente Retrofit

Debemos crear una instancia que nos permitirá usar la Biblioteca Retrofit,

```
class RetrofitClient {  
  
    companion object {  
        private const val URL_BASE =  
            "https://recipe-puppy.p.rapidapi.com"  
        private val retro = RetrofitClient  
  
        fun getRetrofit(): ApiClient {  
            val retrofit = Retrofit.Builder()  
                .baseUrl(URL_BASE)  
  
                .addConverterFactory(GsonConverterFactory.create())  
                .build()  
            return retrofit.create(ApiClient::class.java)  
        }  
    }  
}
```

Interfaz de operaciones

Retrofit utiliza una interface para declarar los métodos a utilizar.

Es posible añadir información extra que el servicio a consumir pueda utilizar.

Es posible utilizar Coroutines para hacer las llamadas al servidor.

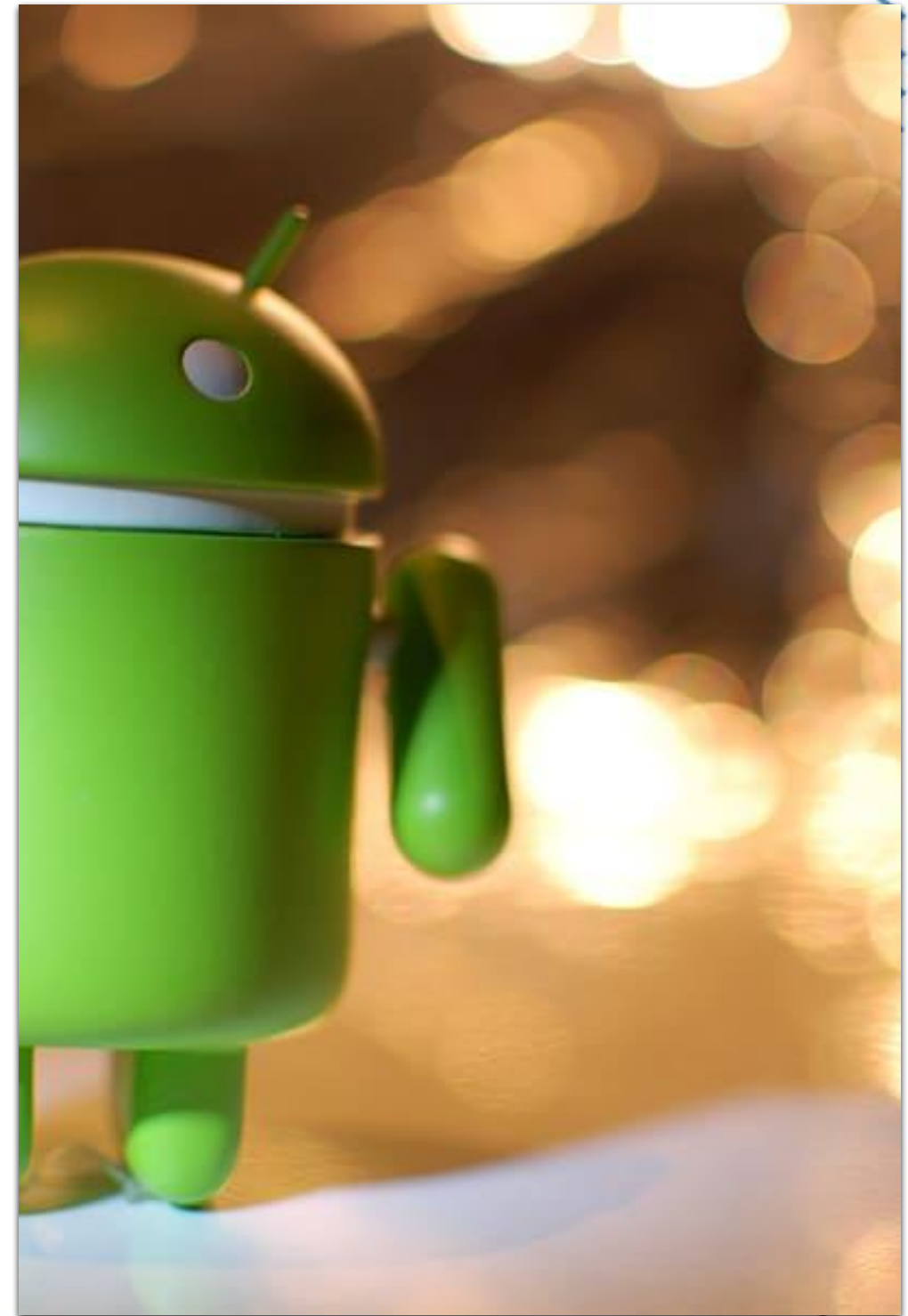
```
interface ApiClient {  
  
    @GET("/")  
    @Headers("x-rapidapi-host: recipe-puppy.p.rapidapi.com",  
            "x-rapidapi-key:  
EiQ7kK3duCmshADAOmGDD7gCtoPBp1IXVX0jsn1fuDYkG  
mBXes")  
    suspend fun getFetchFromServer(): Response<Recipe>  
  
    @GET("/")  
    @Headers("x-rapidapi-host: recipe-puppy.p.rapidapi.com",  
            "x-rapidapi-key:  
EiQ7kK3duCmshADAOmGDD7gCtoPBp1IXVX0jsn1fuDYkG  
mBXes")  
    fun getFetchFromServerEnqueue(): Call<Recipe>  
  
}
```



Retrofit

¿En qué se basa?:

Retrofit se basa en la librería OkHttp de Square, que es una librería de cliente HTTP para Android. Esto aporta ventajas adicionales, incluyendo la gestión de la conectividad de red y la capacidad de realizar solicitudes.

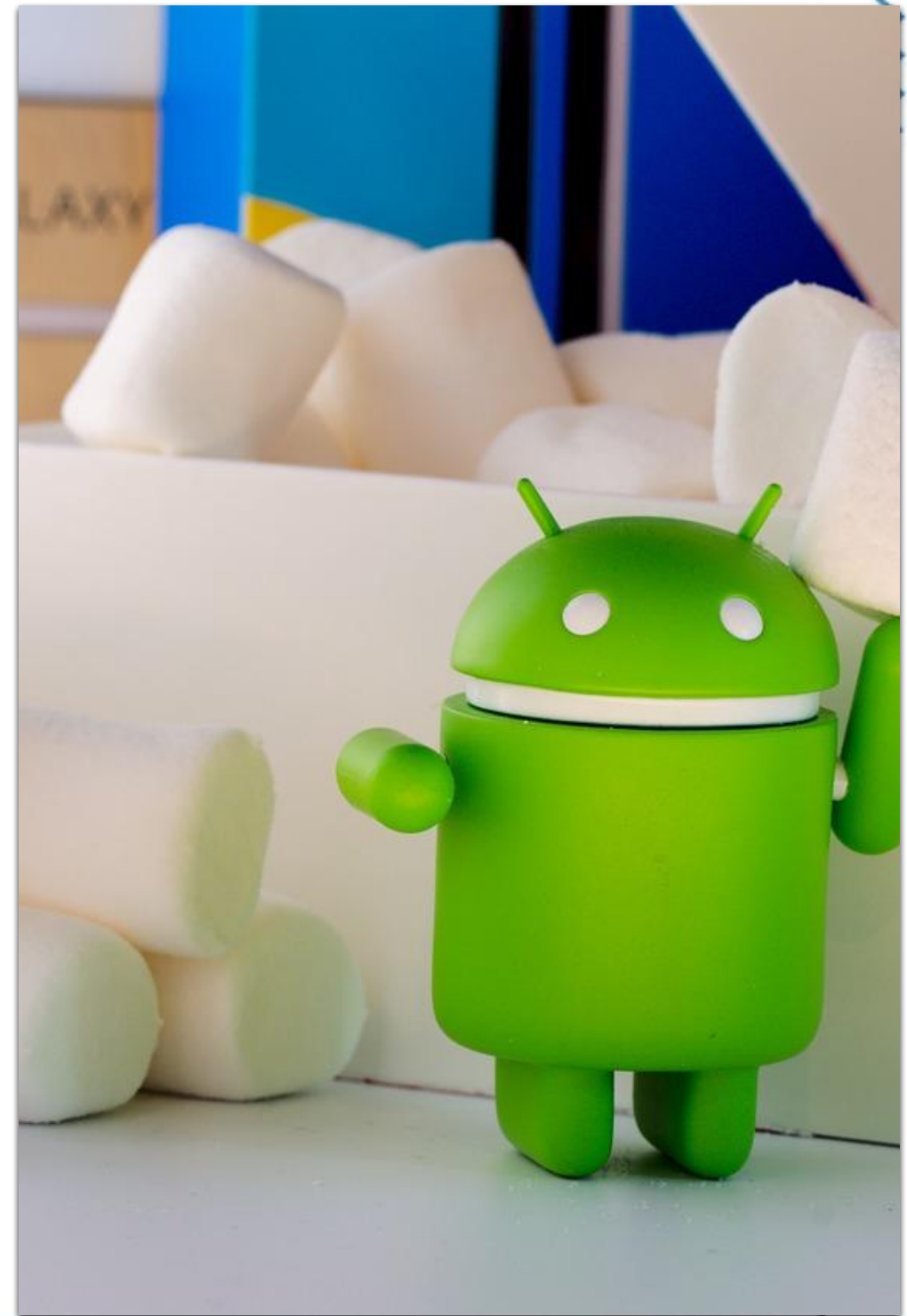




Retrofit

¿Qué más ofrece?:

Proporciona soporte para gestionar diversos tipos de parámetros en las solicitudes, como parámetros de consulta, encabezados personalizados y campos en el cuerpo de la solicitud. La biblioteca se encarga de codificar y enviar estos parámetros al servidor de forma automática.





Retrofit

¿Qué más ofrece?:

Retrofit automatiza la conversión de respuestas del servidor en objetos de tipos de datos específicos, eliminando la necesidad de procesar manualmente respuestas en formato JSON. Utiliza convertidores para mapear los datos directamente a objetos Kotlin o Java, simplificando el manejo y procesamiento de los datos de la API.





Retrofit

¿Qué más ofrece?:

Retrofit utiliza el patrón de Inyección de Dependencias y anotaciones para generar código boilerplate, lo que simplifica su configuración y uso. Además, cuenta con una comunidad activa de desarrolladores que garantiza un soporte constante y ofrece recursos y ejemplos útiles en línea.



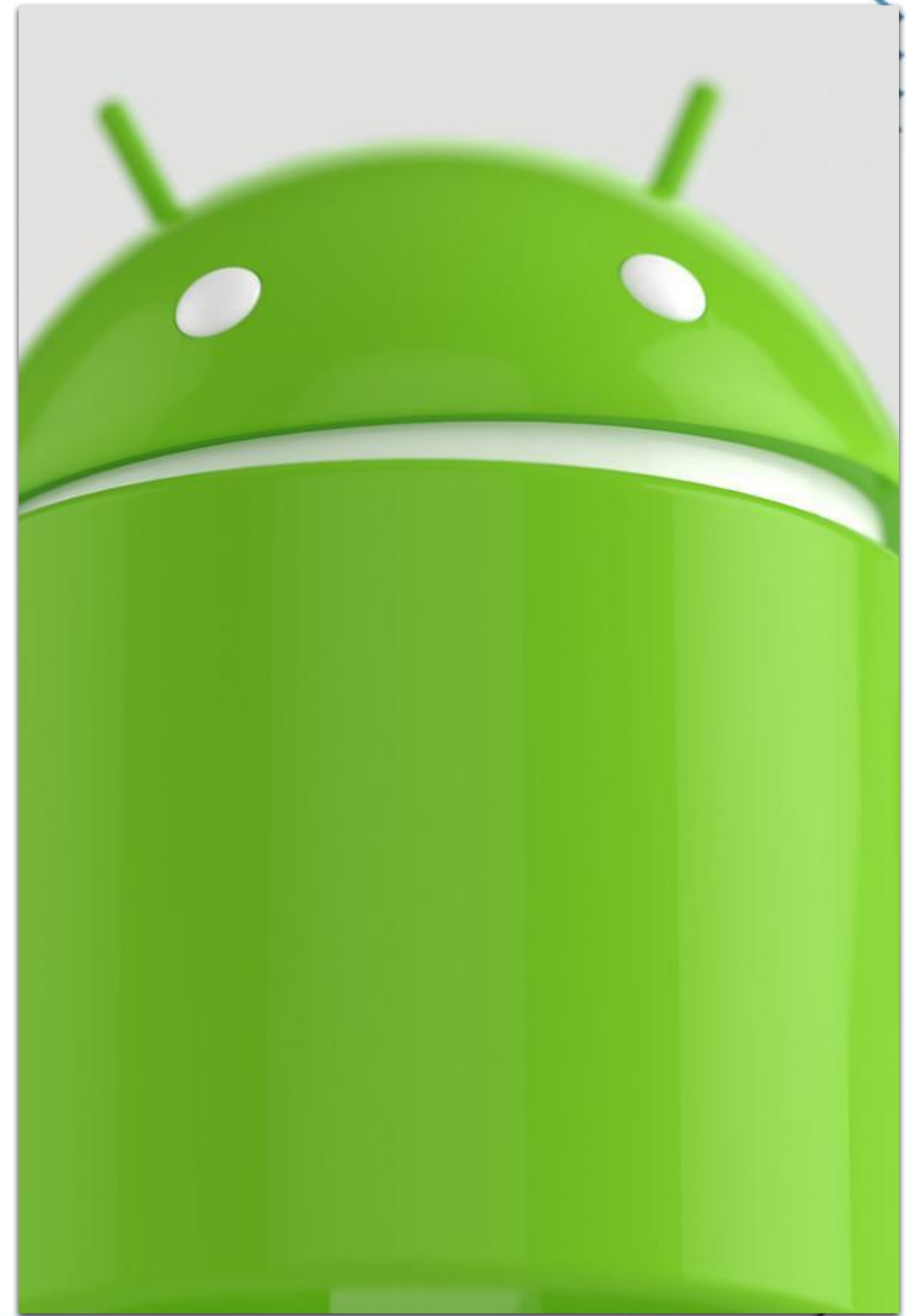


Retrofit

Conclusión:

Retrofit permite a los desarrolladores centrarse en la lógica de la aplicación en lugar de preocuparse por la comunicación de red y la manipulación de datos.

Simplifica la consumición de APIs REST en aplicaciones Android, acelerando el desarrollo y mejorando la calidad de la comunicación entre la aplicación y el servidor.





Anotaciones de Retrofit



Retrofit utiliza varias anotaciones para facilitar su uso, por ejemplo:

```
@GET("users/list")
```

```
@GET("users/list?sort=desc")
```

```
@GET("group/{id}/users")
```

```
Call<List<User>> groupList(@Path("id") int groupId);
```

```
@GET("group/{id}/users")
```

```
Call<List<User>> groupList(@Path("id") int groupId, @Query("sort") String sort);
```

```
@POST("users/new")
```

```
Call<User> createUser(@Body User user);
```

Existen muchas formas de utilizar Retrofit.

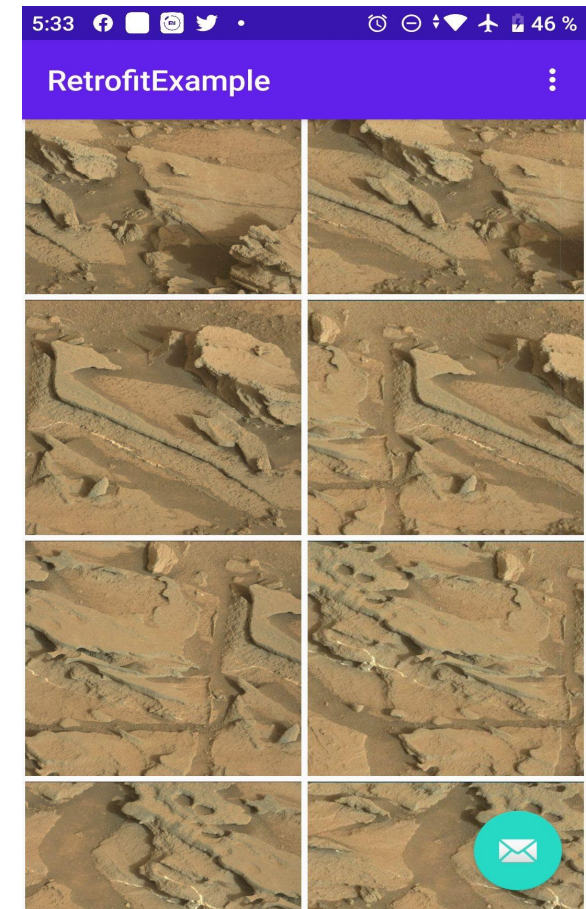
[Documentación.](#)

› Consumo de APIs

Ejercicio: Integrar Retrofit

1. Vamos a conectarnos a un servicio web y traernos los datos hacia nuestra app.
2. Retrofit maneja la conexión y a través de Gson se transformarán los datos a objeto.
3. Mostraremos el listado de objetos en una vista.
4. Tenemos un repositorio con el proyecto inicial, donde ya están construidas las vistas.

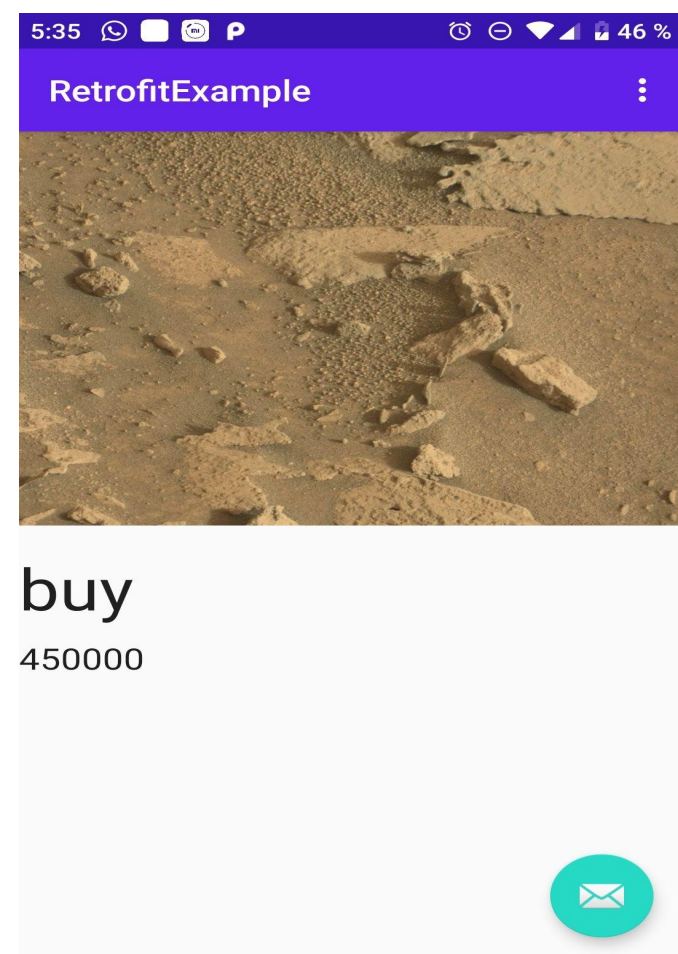
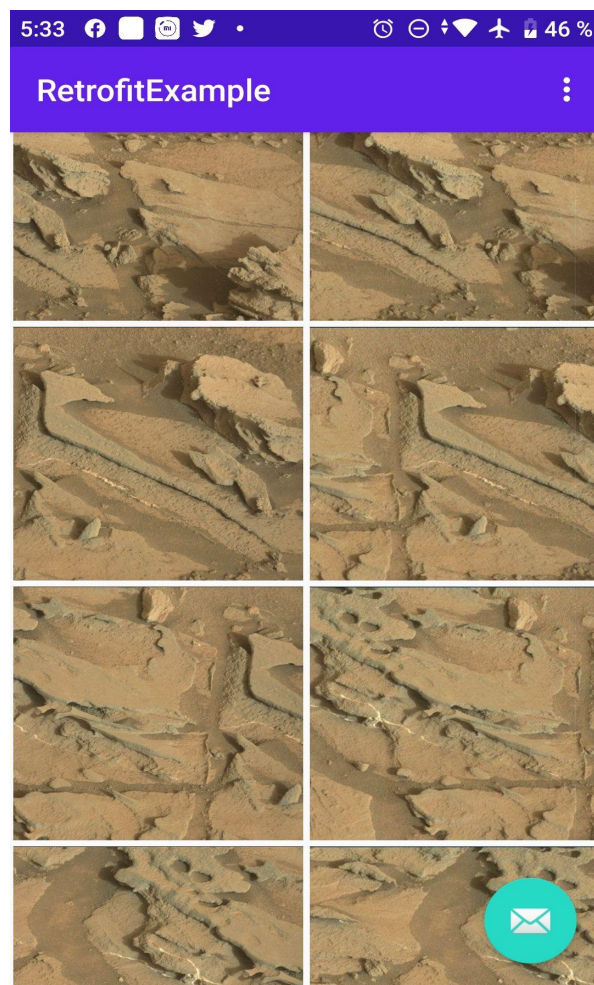
TIEMPO 1:30 HORAS 2 sesiones





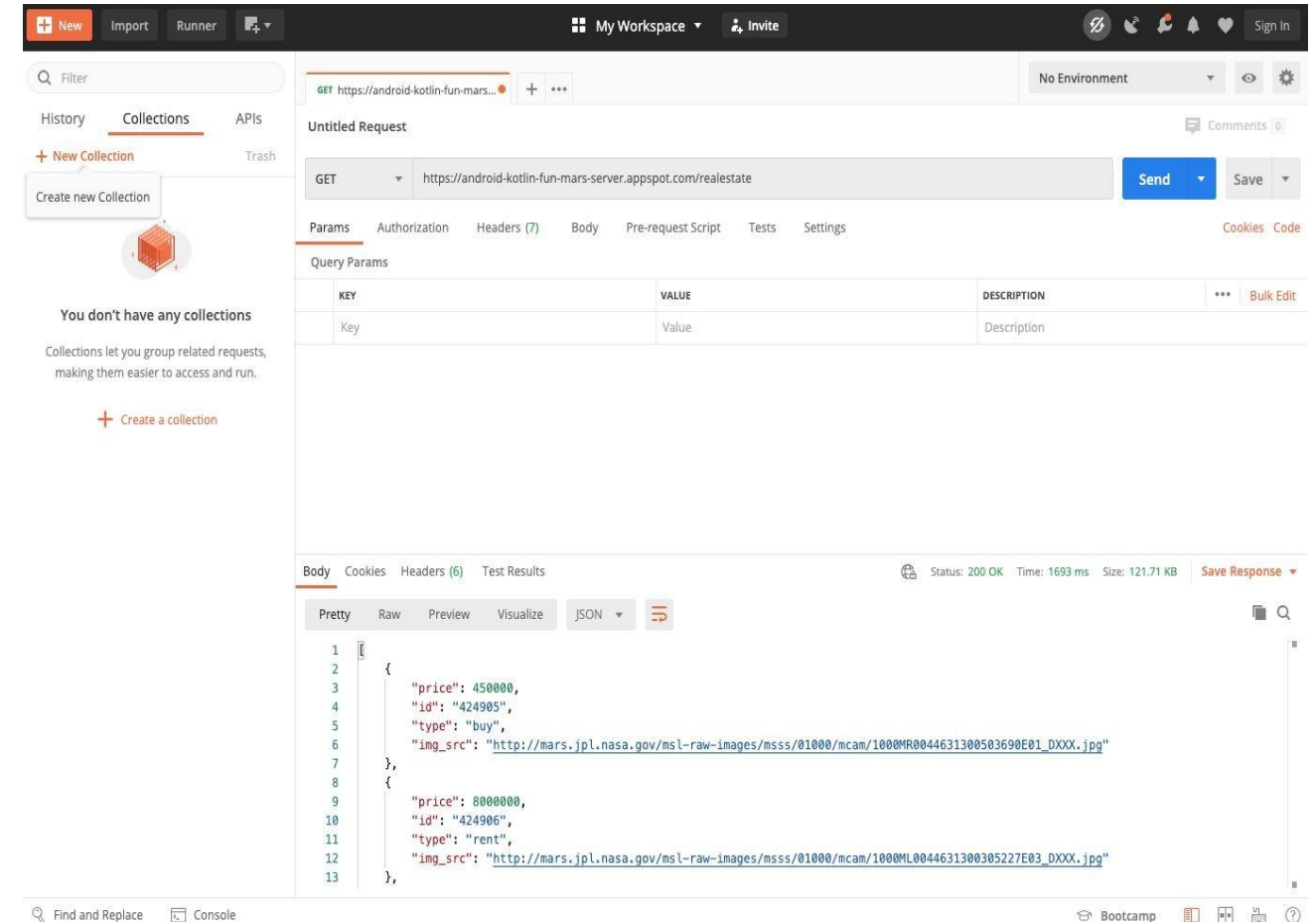
Una vez construida la app debería verse así:

1. Nos conectaremos a una API que nos va a entregar un listado de elementos.
2. Tendremos que analizar los elementos que nos enviaran.
TIP: utiliza POSTMAN
3. <https://android-kotlin-fun-mars-server.appspot.com/>
4. Descarga el proyecto base desde el siguiente repositorio.



LIVECODING: Analizar la respuesta de la API

1. Utiliza Postman para analizar los datos que recibiremos desde la API.
2. Utiliza el Plugin o convertirlo a mano
3. Generar el Pojo para representar los datos.
 - a. URL:
`https://android-kotlin-fun-mars-server.appspot.com/realstate`



LIVECODING: Pojo.

El primer paso es analizar la respuesta del servicio web y construir la clase que instancia el objeto para representarla.

La anotación `@SerializedName` pertenece a Gson y sirven para hacer calzar el nombre del atributo con nuestros campos en Java cuando se está transformando.

`@Expose` es opcional, pero podemos sus argumentos (false) para dejar de convertir algunos atributos.

```
@Entity(tableName =  
    "terrain_table") data  
class Terrain(val price:  
    Long,  
                @PrimaryKey val  
                id: String, val  
                type: String,  
                @SerializedName("img_src")val  
                imgSrc: String)
```

LIVECODING: Interface API

Construir la interface que indicará los métodos a Retrofit.

En este caso solo vamos a utilizar la anotación @GET y le pasaremos el endpoint al que conectarse como argumento.

```
interface ApiInterface {  
  
    @GET("realestate")  
    fun getDataFromApi() :  
        Call<List<Terrain>>
```



LIVECODING: Retrofit Client

Hay que crear una clase que nos devuelva la instancia de Retrofit que vamos a utilizar.

Utilizaremos Gson para la conversión por eso se utiliza el método
.addConverterFactory.

No utilizaremos un adaptador específico, si si así fuera se debe pasar como parametro en
.addCallAdapterFactory.

```
class RetrofitClient {  
  
    companion object {  
        private const val URL_BASE =  
            "https://android-kotlin-fun-mars-server.appspot.com/"  
  
        fun getRetrofitClient() : ApiInterface {  
            val mRetrofit = Retrofit.Builder()  
                .baseUrl(URL_BASE)  
  
                .addConverterFactory(GsonConverterFactory.create())  
                .build()  
            return mRetrofit.create(ApiInterface::class.java)  
        }  
    }  
}
```



LIVECODING: Repository

Para seguir el patrón de arquitectura recomendado, crearemos una clase repository, en donde manejaremos la petición de los datos.

Crearemos una interface llamada remote con el método a implementar.

```
class Repository(private val terrainDao: TerrainDao) {

    private val service =
        RetrofitClient.getRetrofitClient() val mLiveData =
            terrainDao.getAllTerrainsFromDB()

    // La vieja confiable
    fun getDataFromServer() {
        val call = service.getDataFromApi()
        call.enqueue(object : Callback<List<Terrain>>
        {

            override fun onResponse(call: Call<List<Terrain>>,
                response: Response<List<Terrain>>) {
                when(response.code()) {
                    in 200..299 -> CoroutineScope(Dispatchers.IO).launch {
                        response.body()?.let {
                            terrainDao.insertAllTerrains(it)
                        }
                    }
                    in 300..399 -> Log.d("ERROR 300",
                        response.errorBody()?.toString())
                }
            }

            override fun onFailure(call: Call<List<Terrain>>, t: Throwable)
            {
                Log.e("Repository", t.message.toString())
            }
        })
    }
}
```

Review

1. REST web services.
 - a. Un web service te permitirá a través de internet obtener los datos para tu aplicación.
 - b. Es muy común que los web services utilizan la arquitectura [REST](#). Regularmente estos son llamados servicios Restful. Ya que siguen protocolos y estándares web.
 - c. Las llamadas a estos servicios son a través de peticiones web.
 - d. Para usar un web services, la app debe establecer una conexión de red con el servicio, luego debe recibir y modelar los datos en un formato usable por la app.
 - e. Retrofit es una biblioteca que nos permite utilizar servicios web.
 - f. Podemos usar diferentes converters y callFactories en retrofit. En el ejemplo usamos Gson.
 - g. Debes asignarle permisos de internet a tu app y además permite conexiones no seguras en el

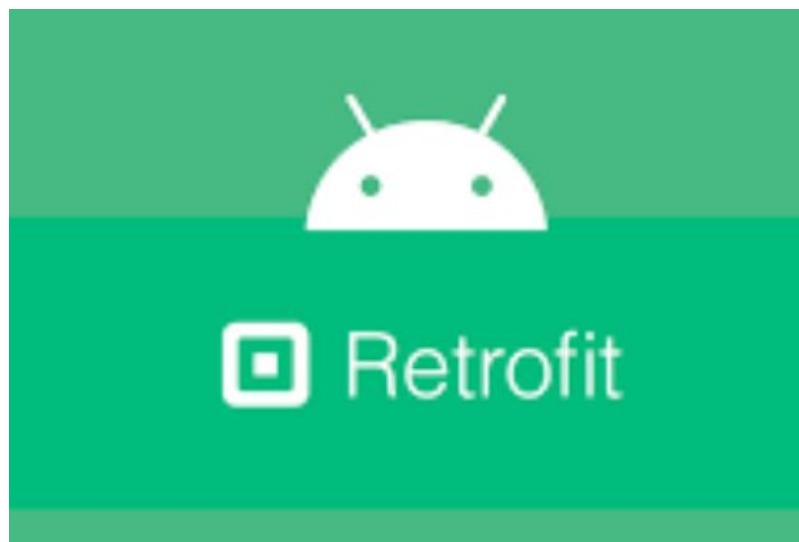
manifest `android:usesCleartextTraffic="true"` . Al interior de
.
`<application>`.



Review.



1. Otros adapter de Retrofit.
 - a. Retrofit nos permite utilizar otras bibliotecas para recibir los datos, por ejemplo RXJava o Coroutines.
 - b. Estas permite recibir los datos en stream u objetos observables. Haciendo más fácil el desempeño de las conexiones.



¿Alguna consulta?



› Evaluación módulo



Evaluación módulo 6

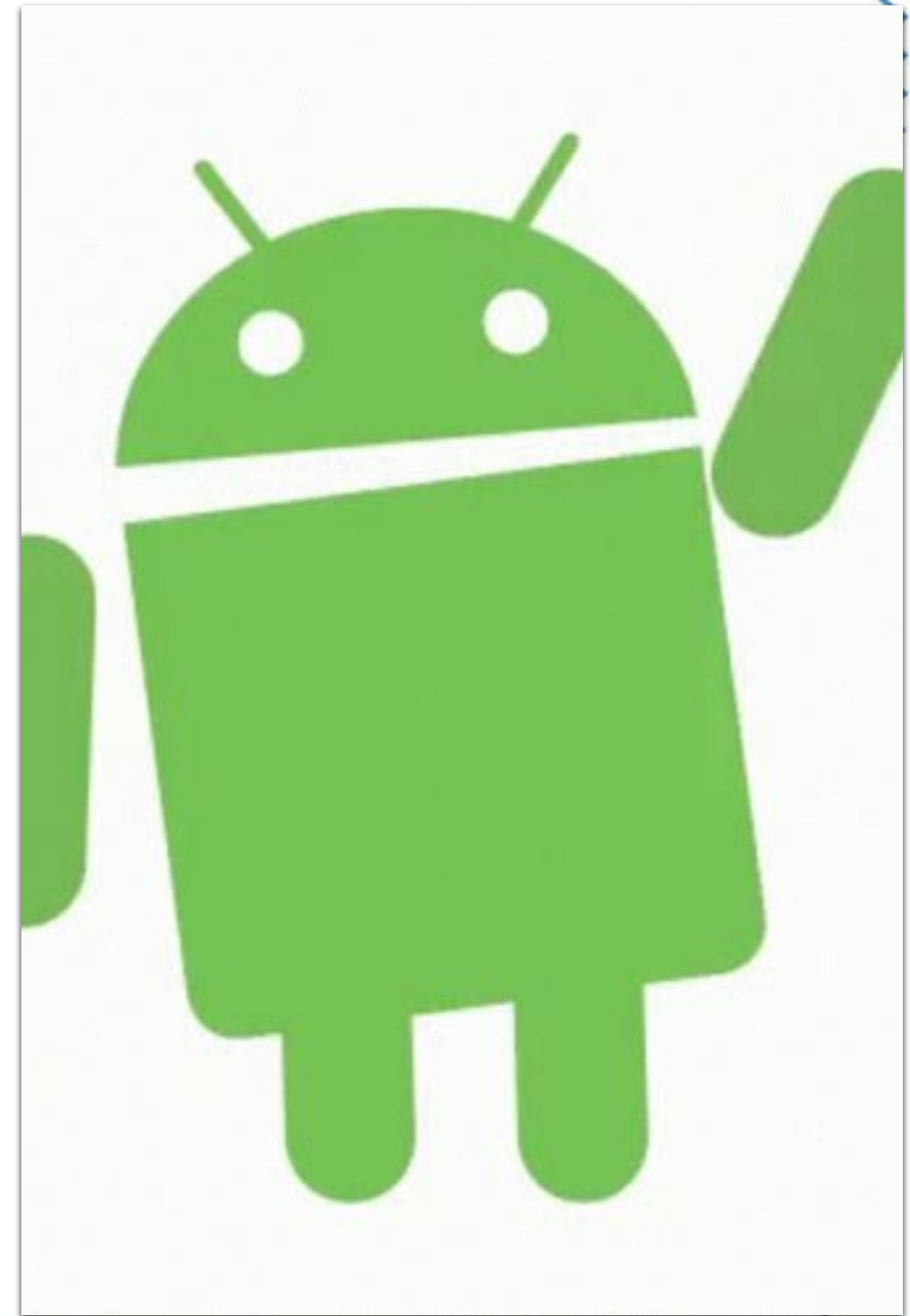
¿En qué consiste?:

En este módulo vamos a estar priorizando completar las funcionalidades de la aplicación en la que estuvimos trabajando en los dos módulos anteriores. Para eso vamos a enfocarnos en agregar dos funcionalidades claves para completar la app: La capacidad de almacenar datos localmente en una base de datos y la capacidad de consumir una API remota que trabajaremos en próximas lecciones.



Situación inicial

En etapas previas, se ha construido la estructura básica de la aplicación, que abarca todas las interfaces necesarias y el flujo de información entre la visualización y la lógica de control. En esta etapa, el objetivo radica en culminar el proceso de desarrollo de la aplicación, incorporando la funcionalidad para establecer conexión con una API REST externa





Nuestros objetivos

El primer objetivo es llevar a cabo la funcionalidad principal de la aplicación, que es permitir a los usuarios efectuar transacciones virtuales. Esto implica implementar la lógica de comunicación con la API REST utilizando Retrofit.





Nuestros objetivos

Otro objetivo importante es implementar la base de datos local utilizando Room. Esto permitirá almacenar datos en el dispositivo del usuario para garantizar un acceso rápido y sin conexión a información relevante, como el historial de transacciones y la información del perfil del usuario.





Requerimientos 🤝

En primer lugar, es esencial brindar a los usuarios la capacidad de registrarse e iniciar sesión de manera segura. Esto implica la implementación de mecanismos de autenticación utilizando técnicas adecuadas, como tokens de seguridad y gestión de sesiones, para asegurar el acceso a la aplicación solo por usuarios autorizados.





Requerimientos 🤝

La visualización de las transacciones realizadas y el seguimiento de los gastos son funcionalidades cruciales. Para lograrlo, se utilizará la librería Retrofit, que permitirá establecer una comunicación fluida con una API REST externa. Esta comunicación facilitará la obtención de los datos necesarios, como detalles de transacciones, para mostrarlos en la interfaz de usuario.





Requerimientos 🤝

La posibilidad de realizar transacciones virtuales requiere una integración sin problemas con la API REST, lo cual será gestionado por Retrofit. Los datos necesarios para realizar transacciones serán enviados a través de solicitudes HTTP, asegurando que los movimientos se realicen de manera exitosa y segura.



Requisitos funcionales

Registro e inicio de sesión: La aplicación debe permitir a los usuarios registrarse y luego iniciar sesión con sus credenciales.

Historial de transacciones: La aplicación debe mostrar un historial de transacciones, detallando la fecha, monto y descripción de cada una.

Realización de transacciones virtuales: Los usuarios deben poder efectuar transacciones virtuales, indicando el monto y una descripción opcional.

Almacenamiento local: La aplicación debe almacenar en una base de datos local el historial de transacciones y los detalles del perfil del usuario para un acceso sin conexión.

Carga de imágenes: La imagen del perfil del usuario debe cargarse y mostrarse adecuadamente en la interfaz.

Requisitos funcionales

Librería Retrofit: Implementar Retrofit para establecer comunicación con la API REST y realizar solicitudes HTTP para obtener y enviar datos.

Librería Room: Utilizar Room para crear una base de datos local que almacene el historial de transacciones y los detalles del perfil del usuario.

Patrón de Arquitectura MVVM: Diseñar la aplicación siguiendo el patrón de arquitectura MVVM, separando la lógica de presentación, la lógica de negocio y los datos.

Integración con Picasso: Utilizar la librería Picasso para cargar y mostrar imágenes.

Gestión de sesiones y autenticación: Implementar mecanismos de gestión de sesiones y autenticación segura para proteger los datos de los usuarios.

Manejo de errores: Implementar manejo de errores adecuado al comunicarse con la API y al realizar operaciones locales, proporcionando mensajes claros al usuario en caso de fallos.

Pruebas unitarias y de integración: Realizar pruebas exhaustivas, incluyendo pruebas unitarias para las distintas capas de la aplicación y pruebas de integración para asegurar el correcto funcionamiento.



Evaluación módulo 6

Entregables

El entregable final será una aplicación de billetera virtual Wallet plenamente funcional y lista para ser utilizada por los usuarios. Este aplicativo se podrá instalar en dispositivos Android a través de un archivo APK o una tienda de aplicaciones.

La aplicación permitirá a los usuarios registrarse, iniciar sesión, visualizar operaciones, efectuar transacciones de envío y recibo de dinero y ver información del perfil.



Recursos

API Wallet.

En próximas clases vamos a estar aprendiendo cómo se usan las APIs en el Android y cuales son las librerías que podemos usar para implementarlo en nuestra aplicaciones.

Para trabajar en esta etapa vamos a usar la siguiente API:

<http://wallet-main.eba-ccwdurgr.us-east-1.elasticbeanstalk.com/api-docs/>

LIVE CODING

Ejemplo en vivo

Demostración:

Mostrar la documentación de la evaluación y resolver dudas sobre la implementación.

Explicar los recursos como la API y avanzar sobre las dudas de integración del proyecto

Tiempo: 30 min

Evaluación Integradora ✨

¿Listos para un nuevo desafío? En esta clase comenzamos a construir en nuestro....

Trabajo Integrador del Módulo 💪

Iremos completándolo progresivamente clase a clase



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?

- ✓ Uso de Api REST
- ✓ HTTP
- ✓ Consumo de APIs
- ✓ Postman
- ✓ Evaluación del módulo
- ✓ Situación inicial
- ✓ Objetivos
- ✓ Requerimientos
- ✓ Hacer Requests HTTP
- ✓ Guardar e importar

< **¡Muchas gracias!** >

